

Banking Management System

A normalised MySQL database for retail banking operations — customers, accounts, transactions, loans, cards, ATMs and branch staff — with an analytics layer built from views, aggregate reporting queries, window functions and stored procedures.

DATABASE ENGINE

MySQL 8.0

InnoDB · utf8mb4

SCHEMA

banking_system

13 tables · 10 views · 14 procedures

SEED DATA

246 rows

20 customers · 25 accounts · 8 branches

CONCEPTS COVERED

DDL · DML · Joins

Views, aggregates, RANK(), stored procedures

WHAT THIS DOCUMENT CONTAINS

Every object in the project is reproduced here in full: the complete DDL for all tables, every `INSERT` statement, all ten view definitions with their executed result sets, eleven analytical dashboard queries with output, all fourteen stored procedures with sample calls, and a code-review section documenting a defect found in the original script together with its correction.

Table of Contents

01	Project Overview Scope, objectives and the engineering skills the schema demonstrates.	2
02	Schema Architecture Entity map, relationship model and the referential integrity plan.	3
03	Database & Table Definitions Steps 1–15: DDL for all thirteen tables plus schema alterations.	5
04	Master & Reference Data Steps 16–18: branches, departments and the customer register.	9
05	Operational Data Load Steps 19–28: accounts, transactions, transfers, loans, cards, ATMs, staff.	11
06	Analytical Views Steps 29–38: ten reporting views with their full result sets.	16
07	Dashboard & Reporting Queries Steps 39–49: aggregation, ranking and window-function analysis.	23
08	Stored Procedures Steps 50–63: fourteen parameterised routines with sample output.	28
09	Code Review & Corrections The duplicate fund_transfers defect, its impact and the fix.	35
10	Results, Setup & Roadmap How to run the project, object inventory and future enhancements.	37

Project Overview

A single MySQL schema that models the day-to-day operations of a multi-branch retail bank, then layers reporting and analytics on top of it.

The **banking_system** database models a retail bank operating eight branches across Maharashtra. It tracks the full customer lifecycle — registration, account opening, deposits and withdrawals, fund transfers, loan disbursement and repayment, card issuance and ATM activity — alongside the internal organisation of branches, departments and employees. On top of that transactional core sits an analytics layer of views, grouped reporting queries and parameterised stored procedures that answer the questions a branch manager or credit officer would actually ask.

13 BASE TABLES	10 REPORTING VIEWS	14 STORED PROCEDURES
246 SEEDED ROWS	18 FOREIGN KEYS	11 DASHBOARD QUERIES

Objectives

- Design a normalised relational schema in which every business entity — branch, customer, account, transaction, loan, card, ATM, employee — owns its own table and is linked through declared foreign keys rather than duplicated columns.
- Enforce data quality at the schema level using primary keys, `UNIQUE` constraints on natural identifiers (account numbers, PAN, Aadhaar, reference numbers), `NOT NULL` requirements and sensible column defaults.
- Populate the schema with a realistic, internally consistent dataset so that every query in the project returns meaningful output instead of empty result sets.
- Abstract recurring multi-table joins into named views, so reporting queries read like plain business questions.
- Answer analytical questions with `GROUP BY` aggregation, ordered ranking and the `RANK()` window function.
- Package customer-level lookups into reusable stored procedures that take a customer ID and return a complete, pre-joined answer.

Functional Modules

Module	Tables	Responsibility
Branch Network	branches, atms	Branch identity, code, city, manager and status; ATM estate mapped to its parent branch.
Customer Register	customers, beneficiaries	Personal and KYC details (DOB, phone, email, Aadhaar, PAN) and each customer's saved payees.
Deposits	accounts, transactions	Savings, Current, Salary and Fixed Deposit accounts with balance, interest rate and full ledger.
Payments	fund_transfers, atm_transactions	Account-to-account movement over UPI, NEFT, RTGS and IMPS, plus ATM cash and enquiry activity.
Lending	loans, loan_payments	Home, Personal, Business, Car and Education loans with tenure, rate and repayment history.
Cards	cards	Debit and credit cards tied to both a customer and a specific account, stored masked.
Workforce	employees, departments	Staff posting, designation, salary and departmental structure across the branch network.

Technology and Techniques

Area	Applied in this project
DDL	<code>CREATE DATABASE</code> , <code>CREATE TABLE</code> , <code>ALTER TABLE ... ADD COLUMN</code> , <code>ADD CONSTRAINT ... FOREIGN KEY</code> , <code>DESCRIBE</code>
Constraints	Primary keys with <code>AUTO_INCREMENT</code> , composite referential links, <code>UNIQUE</code> , <code>NOT NULL</code> , column <code>DEFAULT</code> values including <code>CURRENT_DATE</code> and <code>CURRENT_TIMESTAMP</code>
DML	Multi-row <code>INSERT</code> loads for all thirteen tables
Joins	<code>INNER JOIN</code> , <code>LEFT JOIN</code> , and self-referencing double joins (sender and receiver of the same transfer resolved from one table)
Views	Ten views that flatten two-, three- and four-table joins into reportable result sets
Aggregation	<code>COUNT</code> , <code>SUM</code> , <code>AVG</code> , <code>COUNT(DISTINCT ...)</code> , <code>COALESCE</code> , <code>GROUP BY</code> , <code>HAVING</code>
Window functions	<code>RANK()</code> <code>OVER (ORDER BY ...)</code> for balance and loan-exposure league tables
Procedures	<code>DELIMITER</code> blocks, <code>IN</code> parameters, <code>DROP PROCEDURE IF EXISTS</code> for idempotent redeployment
Data quality	Duplicate-detection query using <code>GROUP BY ... HAVING COUNT(*) > 1</code>

Schema Architecture

Thirteen tables arranged in three tiers: master entities, the account and loan core, and the activity tables that record what actually happened.

The schema is deliberately shallow and wide rather than deeply nested. Three master entities — `branches`, `customers` and `departments` — own no foreign keys of their own and can be loaded first. Everything else hangs off them. The two chains that carry most of the reporting weight are `customers → accounts → transactions` and `customers → loans → loan_payments`.

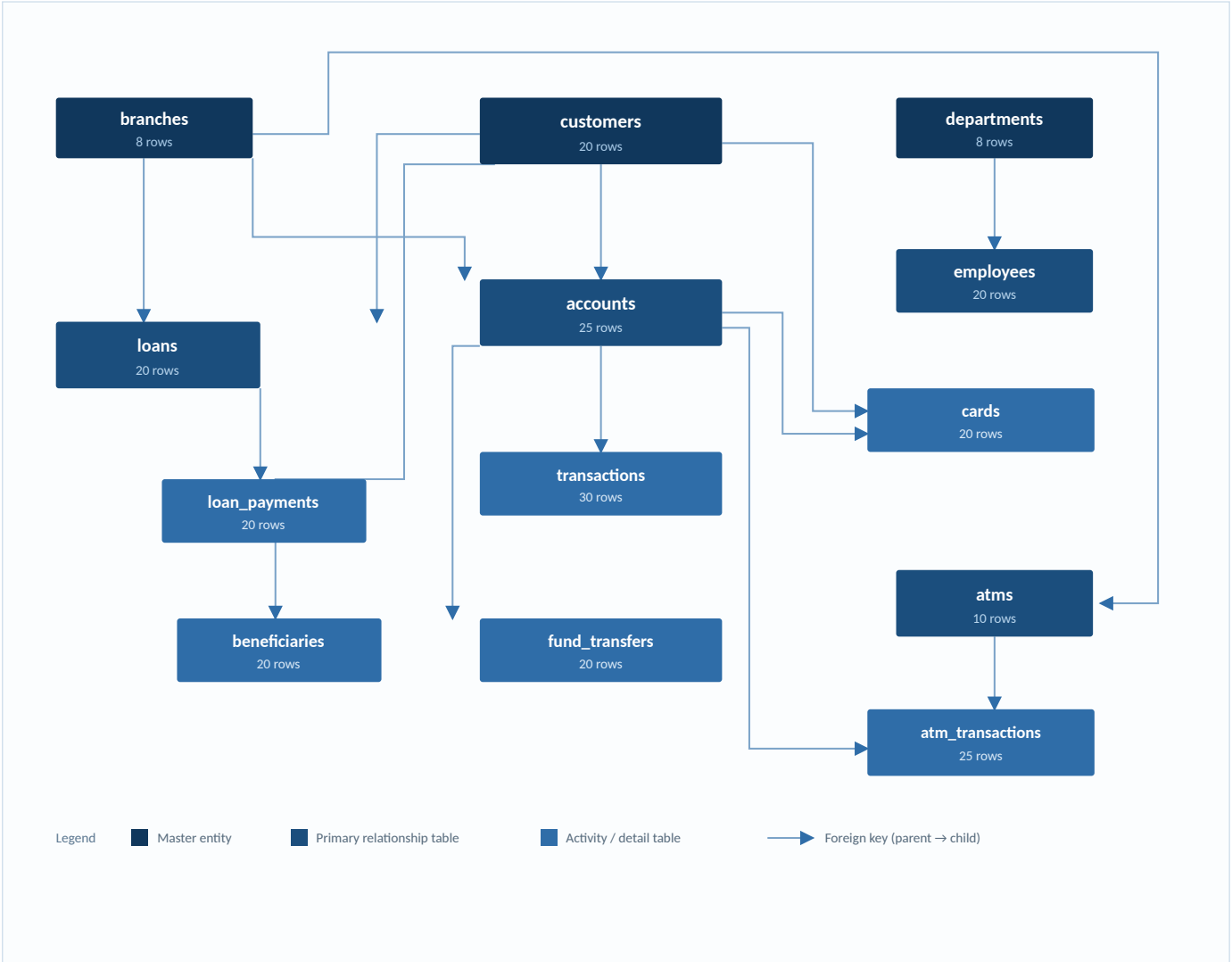


Figure 2.1 — Entity relationship map of the banking_system schema. Arrows point from the parent table to the child table holding the foreign key.

Referential Integrity Map

Eighteen foreign keys bind the schema together. Two of them are unusual and worth noting: `fund_transfers` references `accounts` twice, once as sender and once as receiver, and `cards` references both the customer and the specific account a card draws on, so a card can never be issued against another customer's account.

Child table	Foreign key column(s)	References
accounts	customer_id, branch_id	customers, branches
transactions	account_id	accounts
fund_transfers	sender_account_id, receiver_account_id	accounts (twice)
loans	customer_id, branch_id	customers, branches
loan_payments	loan_id	loans
cards	customer_id, account_id	customers, accounts
atms	branch_id	branches
atm_transactions	atm_id, account_id	atms, accounts
employees	branch_id, department_id	branches, departments
beneficiaries	customer_id	customers

Build Order

Because every foreign key is declared inline at creation time, the tables must be created in dependency order. The one exception is `employees.department_id`, which is added afterwards with `ALTER TABLE` — a deliberate demonstration of evolving a live schema rather than rebuilding it.

Order	Objects created	Why here
Tier 1	branches, customers, departments	No outbound dependencies; safe to create and load first.
Tier 2	accounts, loans, atms, employees, beneficiaries	Depend only on Tier 1 master entities.
Tier 3	transactions, fund_transfers, loan_payments, cards, atm_transactions	Depend on Tier 2; these hold the activity history.
Tier 4	10 views, 14 procedures	Read-only layer; created once all base tables are populated.

Design Conventions

- **Surrogate keys everywhere.** Every table uses an `INT AUTO_INCREMENT` primary key, keeping joins narrow and stable even if a natural identifier is later corrected.
- **Natural keys kept unique.** Account numbers, reference numbers, phone numbers, emails, Aadhaar, PAN and branch codes all carry `UNIQUE` constraints, so the database rejects duplicates rather than relying on application code.
- **Money as DECIMAL.** All monetary columns are `DECIMAL(15,2)` and rates `DECIMAL(5,2)`; floating point is never used for currency.
- **Soft status columns.** Each entity carries a `status` column defaulting to `'Active'`, so records are retired rather than deleted and history stays intact.
- **Auditable references.** Every financial event — transaction, transfer, loan payment, ATM operation — carries a unique `reference_number` for reconciliation.

Database and Table Definitions

Steps 1–15 of the script: creating the schema and all thirteen base tables, then extending the employee table with a departmental foreign key.

The statements below are reproduced exactly as executed. Each table is preceded by a short note on the columns that carry business meaning, so the DDL can be read without cross-referencing.

Step 1 — Create and select the schema

STEP 1 — SCHEMA CREATION

```
CREATE DATABASE banking_system;
USE banking_system;
```

Step 2 — branches

The root of the network. `branch_code` is the natural key a bank actually quotes (MUM001, PUN001) and is therefore unique, while `branch_id` is the surrogate key used for joins.

STEP 2 — BRANCHES

```
CREATE TABLE branches (
  branch_id      INT PRIMARY KEY AUTO_INCREMENT,
  branch_name    VARCHAR(100) NOT NULL,
  branch_code    VARCHAR(20) UNIQUE NOT NULL,
  city           VARCHAR(50) NOT NULL,
  state          VARCHAR(50) NOT NULL,
  manager_name   VARCHAR(100),
  contact_number VARCHAR(15),
  opening_date   DATE,
  status         VARCHAR(20) DEFAULT 'Active'
);
```

Step 3 — customers

Holds identity and KYC data. Four columns are unique: `phone_number`, `email`, `aadhaar_number` and `pan_number` — the database itself prevents a duplicate customer record being created. `registration_date` defaults to the current date.

STEP 3 — CUSTOMERS

```
CREATE TABLE customers (
  customer_id    INT PRIMARY KEY AUTO_INCREMENT,
  full_name      VARCHAR(100) NOT NULL,
  date_of_birth  DATE,
  gender         VARCHAR(10),
  phone_number   VARCHAR(15) UNIQUE NOT NULL,
  email          VARCHAR(100) UNIQUE,
  address        VARCHAR(255),
  city           VARCHAR(50),
  state          VARCHAR(50),
  aadhaar_number VARCHAR(20) UNIQUE,
  pan_number     VARCHAR(20) UNIQUE,
  registration_date DATE DEFAULT (CURRENT_DATE),
  status         VARCHAR(20) DEFAULT 'Active'
);
```

Step 4 — accounts

The hinge of the schema. An account belongs to exactly one customer and is held at exactly one branch, so both foreign keys are `NOT NULL`. A customer may hold several accounts of different types — in the seeded data five customers do.

STEP 4 — ACCOUNTS

```
CREATE TABLE accounts (
  account_id     INT PRIMARY KEY AUTO_INCREMENT,
  customer_id    INT NOT NULL,
  branch_id     INT NOT NULL,
  account_number VARCHAR(20) UNIQUE NOT NULL,
  account_type   VARCHAR(30) NOT NULL,
  balance        DECIMAL(15,2) DEFAULT 0.00,
  interest_rate  DECIMAL(5,2) DEFAULT 0.00,
  opening_date   DATE DEFAULT (CURRENT_DATE),
  status         VARCHAR(20) DEFAULT 'Active',
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
  FOREIGN KEY (branch_id)   REFERENCES branches(branch_id)
);
```

Step 5 — transactions

A single-entry ledger of deposits, withdrawals and transfers against one account. `transaction_date` defaults to `CURRENT_TIMESTAMP`, and `reference_number` is unique so the same slip can never be posted twice.

STEP 5 — TRANSACTIONS

```
CREATE TABLE transactions (
  transaction_id INT PRIMARY KEY AUTO_INCREMENT,
  account_id INT NOT NULL,
  transaction_type VARCHAR(30) NOT NULL,
  amount DECIMAL(15,2) NOT NULL,
  transaction_date DATETIME DEFAULT CURRENT_TIMESTAMP,
  description VARCHAR(255),
  reference_number VARCHAR(50) UNIQUE,
  FOREIGN KEY (account_id) REFERENCES accounts(account_id)
);
```

Step 6 — fund_transfers

The most interesting table in the schema: it references `accounts` twice from a single row, once for the sender and once for the receiver. Resolving both sides to customer names later requires joining `accounts` and `customers` twice under different aliases.

STEP 6 — FUND_TRANSFERS

```
CREATE TABLE fund_transfers (
  transfer_id INT PRIMARY KEY AUTO_INCREMENT,
  sender_account_id INT NOT NULL,
  receiver_account_id INT NOT NULL,
  amount DECIMAL(15,2) NOT NULL,
  transfer_date DATETIME DEFAULT CURRENT_TIMESTAMP,
  transfer_mode VARCHAR(30),
  transfer_status VARCHAR(20) DEFAULT 'Completed',
  reference_number VARCHAR(50) UNIQUE,
  FOREIGN KEY (sender_account_id) REFERENCES accounts(account_id),
  FOREIGN KEY (receiver_account_id) REFERENCES accounts(account_id)
);
```

NOTE

This table is created a second time later in the original script (Step 21A), which fails with `ERROR 1050`. The defect and its correction are documented in full in Section 09.

Step 7 — loans

A loan is issued to a customer at a branch. Tenure is stored in months and the interest rate as `DECIMAL(5,2)`, so 8.50% is stored exactly rather than approximated.

STEP 7 — LOANS

```
CREATE TABLE loans (
  loan_id INT PRIMARY KEY AUTO_INCREMENT,
  customer_id INT NOT NULL,
  branch_id INT NOT NULL,
  loan_type VARCHAR(50) NOT NULL,
  loan_amount DECIMAL(15,2) NOT NULL,
  interest_rate DECIMAL(5,2) NOT NULL,
  tenure_months INT NOT NULL,
  start_date DATE,
  end_date DATE,
  loan_status VARCHAR(30) DEFAULT 'Active',
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
  FOREIGN KEY (branch_id) REFERENCES branches(branch_id)
);
```

Step 8 — loan_payments

Each repayment points back at its loan. Summing this table against `loans.loan_amount` is what produces the outstanding-balance report in Section 06.

STEP 8 — LOAN_PAYMENTS

```
CREATE TABLE loan_payments (
  payment_id INT PRIMARY KEY AUTO_INCREMENT,
  loan_id INT NOT NULL,
  payment_date DATE NOT NULL,
  payment_amount DECIMAL(15,2) NOT NULL,
  payment_method VARCHAR(30),
  payment_status VARCHAR(20) DEFAULT 'Paid',
  reference_number VARCHAR(50) UNIQUE,
  FOREIGN KEY (loan_id) REFERENCES loans(loan_id)
);
```

Step 9 — cards

Card numbers are stored masked (`XXXX-XXXX-XXXX-1001`) rather than in full — the schema is modelled so that sensitive data is never persisted in the clear.

STEP 9 — CARDS

```
CREATE TABLE cards (
  card_id          INT PRIMARY KEY AUTO_INCREMENT,
  customer_id      INT NOT NULL,
  account_id       INT NOT NULL,
  card_type        VARCHAR(20) NOT NULL,
  card_number_masked VARCHAR(25) NOT NULL,
  issue_date       DATE,
  expiry_date      DATE,
  card_status      VARCHAR(20) DEFAULT 'Active',
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
  FOREIGN KEY (account_id) REFERENCES accounts(account_id)
);
```

Step 10 — atms

Every ATM is owned by a branch, which lets ATM activity be rolled up to branch level without any extra mapping table.

STEP 10 — ATMS

```
CREATE TABLE atms (
  atm_id          INT PRIMARY KEY AUTO_INCREMENT,
  branch_id       INT NOT NULL,
  atm_location    VARCHAR(150) NOT NULL,
  city            VARCHAR(50) NOT NULL,
  installation_date DATE,
  status          VARCHAR(20) DEFAULT 'Active',
  FOREIGN KEY (branch_id) REFERENCES branches(branch_id)
);
```

Step 11 — atm_transactions

Records withdrawals, deposits and balance enquiries. Because an enquiry moves no money, `amount` defaults to `0.00` rather than being nullable.

STEP 11 — ATM_TRANSACTIONS

```
CREATE TABLE atm_transactions (
  atm_transaction_id INT PRIMARY KEY AUTO_INCREMENT,
  atm_id             INT NOT NULL,
  account_id         INT NOT NULL,
  transaction_type    VARCHAR(30) NOT NULL,
  amount             DECIMAL(15,2) DEFAULT 0.00,
  transaction_date    DATETIME DEFAULT CURRENT_TIMESTAMP,
  status             VARCHAR(20) DEFAULT 'Successful',
  reference_number    VARCHAR(50) UNIQUE,
  FOREIGN KEY (atm_id) REFERENCES atms(atm_id),
  FOREIGN KEY (account_id) REFERENCES accounts(account_id)
);
```

Step 12 — employees

Created initially with a free-text `department` column and a branch foreign key. The normalised `department_id` link is added in Step 15.

STEP 12 — EMPLOYEES

```
CREATE TABLE employees (
  employee_id      INT PRIMARY KEY AUTO_INCREMENT,
  branch_id        INT NOT NULL,
  employee_name     VARCHAR(100) NOT NULL,
  designation       VARCHAR(50),
  department        VARCHAR(50),
  phone_number     VARCHAR(15),
  email            VARCHAR(100) UNIQUE,
  joining_date      DATE,
  salary            DECIMAL(12,2),
  status           VARCHAR(20) DEFAULT 'Active',
  FOREIGN KEY (branch_id) REFERENCES branches(branch_id)
);
```

Step 13 — beneficiaries

Saved payees belonging to a customer. These are external accounts at other banks, so they store an account number and IFSC code as plain text rather than a foreign key.

STEP 13 — BENEFICIARIES

```
CREATE TABLE beneficiaries (
  beneficiary_id    INT PRIMARY KEY AUTO_INCREMENT,
  customer_id       INT NOT NULL,
  beneficiary_name   VARCHAR(100) NOT NULL,
  beneficiary_account_number VARCHAR(20) NOT NULL,
  beneficiary_bank_name VARCHAR(100),
  ifsc_code         VARCHAR(20),
  relationship      VARCHAR(50),
  added_date        DATE DEFAULT (CURRENT_DATE),
  status            VARCHAR(20) DEFAULT 'Active',
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

Step 14 — departments

A small lookup table with a unique department name, created after `employees` and wired to it in the next step.

STEP 14 — DEPARTMENTS

```
CREATE TABLE departments (  
  department_id INT PRIMARY KEY AUTO_INCREMENT,  
  department_name VARCHAR(100) UNIQUE NOT NULL,  
  description VARCHAR(255),  
  status VARCHAR(20) DEFAULT 'Active'  
);
```

Step 15 — Schema alteration: linking employees to departments

Rather than dropping and recreating `employees`, the column and its constraint are added in place. This is the pattern used for live schema migration, and it is why the script inspects the table with `DESCRIBE` first.

STEP 15 — ALTER TABLE EMPLOYEES

```
DESCRIBE employees;  
  
-- Step 15A — add the column  
ALTER TABLE employees  
ADD COLUMN department_id INT;  
  
-- Step 15B — add the referential constraint  
ALTER TABLE employees  
ADD CONSTRAINT fk_employee_department  
FOREIGN KEY (department_id)  
REFERENCES departments(department_id);
```

Table Inventory

Table	Rows	Unique keys	Primary purpose
branches	8	branch_code	Branch network master
customers	20	phone, email, aadhaar, pan	Customer identity and KYC
departments	8	department_name	Organisational structure
accounts	25	account_number	Deposit accounts and balances
loans	20	—	Loan book
atms	10	—	ATM estate
employees	20	email	Staff register
beneficiaries	20	—	Saved external payees
transactions	30	reference_number	Account ledger
fund_transfers	20	reference_number	Inter-account payments
loan_payments	20	reference_number	Loan repayments
cards	20	—	Debit and credit cards
atm_transactions	25	reference_number	ATM activity log

Master and Reference Data

Steps 16–18: loading the three tables that own no foreign keys — the branch network, the departmental structure and the customer register.

These loads must run first. Every subsequent insert depends on the identifiers generated here, and because each foreign key is enforced, attempting to load out of order would be rejected by the engine rather than silently creating orphan rows.

Step 16 — Branch network (8 rows)

Eight branches across Maharashtra, opened between 2016 and 2021. The branch code follows a city prefix convention, with three Mumbai branches numbered MUM001–MUM003.

STEP 16 — INSERT INTO BRANCHES

INSERT INTO branches
(branch_name, branch_code, city, state, manager_name, contact_number, opening_date, status)
VALUES
(
'Mumbai Central Branch', 'MUM001', 'Mumbai', 'Maharashtra', 'Rahul Sharma', '9000000001', '2018-04-10', 'Active'),
(
'Andheri Branch', 'MUM002', 'Mumbai', 'Maharashtra', 'Priya Mehta', '9000000002', '2019-06-15', 'Active'),
(
'Pune Main Branch', 'PUN001', 'Pune', 'Maharashtra', 'Amit Patil', '9000000003', '2017-02-20', 'Active'),
(
'Nashik Branch', 'NSK001', 'Nashik', 'Maharashtra', 'Neha Joshi', '9000000004', '2020-09-12', 'Active'),
(
'Nagpur Branch', 'NAG001', 'Nagpur', 'Maharashtra', 'Vikas Rao', '9000000005', '2016-11-05', 'Active'),
(
'Thane Branch', 'THN001', 'Thane', 'Maharashtra', 'Sneha Kulkarni', '9000000006', '2021-01-18', 'Active'),
(
'Bandra Branch', 'MUM003', 'Mumbai', 'Maharashtra', 'Karan Shah', '9000000007', '2018-08-25', 'Active'),
(
'Aurangabad Branch', 'AUR001', 'Aurangabad', 'Maharashtra', 'Pooja Deshmukh', '9000000008', '2019-12-10', 'Active');)

LOADED DATA — BRANCHES

branch_id	branch_name	branch_code	city	manager_name	opening_date	status
1	Mumbai Central Branch	MUM001	Mumbai	Rahul Sharma	2018-04-10	Active
2	Andheri Branch	MUM002	Mumbai	Priya Mehta	2019-06-15	Active
3	Pune Main Branch	PUN001	Pune	Amit Patil	2017-02-20	Active
4	Nashik Branch	NSK001	Nashik	Neha Joshi	2020-09-12	Active
5	Nagpur Branch	NAG001	Nagpur	Vikas Rao	2016-11-05	Active
6	Thane Branch	THN001	Thane	Sneha Kulkarni	2021-01-18	Active
7	Bandra Branch	MUM003	Mumbai	Karan Shah	2018-08-25	Active
8	Aurangabad Branch	AUR001	Aurangabad	Pooja Deshmukh	2019-12-10	Active

Step 17 — Departments (8 rows)

The departmental lookup that `employees.department_id` points at. Department names are unique, so the same department can never be entered twice under a different spelling.

STEP 17 — INSERT INTO DEPARTMENTS

INSERT INTO departments
(department_name, description, status)
VALUES
(
'Accounts', 'Handles customer accounts and financial records', 'Active'),
(
'Loans', 'Handles loan applications and loan management', 'Active'),
(
'Customer Service', 'Handles customer queries and support', 'Active'),
(
'Operations', 'Handles daily banking operations', 'Active'),
(
'IT', 'Handles banking technology and systems', 'Active'),
(
'Human Resources', 'Handles employee management and recruitment', 'Active'),
(
'Risk Management', 'Handles financial and operational risks', 'Active'),
(
'Audit', 'Handles internal banking audits', 'Active');)

LOADED DATA — DEPARTMENTS

department_id	department_name	description	status
1	Accounts	Handles customer accounts and financial records	Active
2	Loans	Handles loan applications and loan management	Active
3	Customer Service	Handles customer queries and support	Active
4	Operations	Handles daily banking operations	Active
5	IT	Handles banking technology and systems	Active
6	Human Resources	Handles employee management and recruitment	Active
7	Risk Management	Handles financial and operational risks	Active
8	Audit	Handles internal banking audits	Active

Step 18 — Customer register (20 rows)

Twenty customers registered across 2024, each with a unique phone number, email, Aadhaar and PAN. The Aadhaar and PAN values are deliberately synthetic placeholders (`DUMMYA001`, `DUMPA001`) so that no real identity document format is reproduced in a portfolio project.

STEP 18 — INSERT INTO CUSTOMERS

INSERT INTO customers
(full_name, date_of_birth, gender, phone_number, email, address, city, state, aadhaar_number, pan_number, registration_date, status)
VALUES
(
'Aarav Mehta', '1995-04-12', 'Male', '9100000001', 'aarav@example.com', 'MG Road', 'Mumbai', 'Maharashtra', 'DUMMYA001', 'DUMPA001', '2024-01-10', 'Active'),
)

('Priya Sharma', '1997-08-22', 'Female', '9100000002', 'priya@example.com', 'Andheri West', 'Mumbai', 'Maharashtra', 'DUMMYA002', 'DUMPA002', '2024-01-15', 'Active'),	('Rohan Patil', '1993-02-18', 'Male', '9100000003', 'rohan@example.com', 'Kothrud', 'Pune', 'Maharashtra', 'DUMMYA003', 'DUMPA003', '2024-02-05', 'Active'),	('Sneha Joshi', '1998-11-30', 'Female', '9100000004', 'sneha@example.com', 'College Road', 'Nashik', 'Maharashtra', 'DUMMYA004', 'DUMPA004', '2024-02-20', 'Active'),	('Vikas Rao', '1990-06-14', 'Male', '9100000005', 'vikas@example.com', 'Civil Lines', 'Nagpur', 'Maharashtra', 'DUMMYA005', 'DUMPA005', '2024-03-02', 'Active'),	('Ananya Shah', '1996-09-05', 'Female', '9100000006', 'ananya@example.com', 'Bandra West', 'Mumbai', 'Maharashtra', 'DUMMYA006', 'DUMPA006', '2024-03-18', 'Active'),	('Karan Desai', '1992-12-10', 'Male', '9100000007', 'karan@example.com', 'Thane West', 'Thane', 'Maharashtra', 'DUMMYA007', 'DUMPA007', '2024-04-01', 'Active'),	('Neha Kulkarni', '1999-03-25', 'Female', '9100000008', 'neha@example.com', 'Camp Area', 'Pune', 'Maharashtra', 'DUMMYA008', 'DUMPA008', '2024-04-15', 'Active'),	('Rahul Verma', '1994-07-19', 'Male', '9100000009', 'rahul@example.com', 'Powai', 'Mumbai', 'Maharashtra', 'DUMMYA009', 'DUMPA009', '2024-05-01', 'Active'),	('Pooja Deshmukh', '1997-01-28', 'Female', '9100000010', 'pooja@example.com', 'CIDCO', 'Aurangabad', 'Maharashtra', 'DUMMYA010', 'DUMPA010', '2024-05-16', 'Active'),	('Aditya Singh', '1991-05-16', 'Male', '9100000011', 'aditya@example.com', 'Vashi', 'Mumbai', 'Maharashtra', 'DUMMYA011', 'DUMPA011', '2024-06-05', 'Active'),	('Riya Kapoor', '1998-10-11', 'Female', '9100000012', 'riya@example.com', 'Ghatkopar', 'Mumbai', 'Maharashtra', 'DUMMYA012', 'DUMPA012', '2024-06-20', 'Active'),	('Siddharth Nair', '1989-08-07', 'Male', '9100000013', 'siddharth@example.com', 'Aundh', 'Pune', 'Maharashtra', 'DUMMYA013', 'DUMPA013', '2024-07-02', 'Active'),	('Isha Malhotra', '1996-02-14', 'Female', '9100000014', 'isha@example.com', 'Mulund', 'Mumbai', 'Maharashtra', 'DUMMYA014', 'DUMPA014', '2024-07-18', 'Active'),	('Manish Gupta', '1988-11-21', 'Male', '9100000015', 'manish@example.com', 'Nashik Road', 'Nashik', 'Maharashtra', 'DUMMYA015', 'DUMPA015', '2024-08-01', 'Active'),	('Kavya Iyer', '1995-06-03', 'Female', '9100000016', 'kavya@example.com', 'Dharampeth', 'Nagpur', 'Maharashtra', 'DUMMYA016', 'DUMPA016', '2024-08-15', 'Active'),	('Arjun Rao', '1993-09-27', 'Male', '9100000017', 'arjun@example.com', 'Dadar', 'Mumbai', 'Maharashtra', 'DUMMYA017', 'DUMPA017', '2024-09-01', 'Active'),	('Meera Patil', '1999-12-19', 'Female', '9100000018', 'meera@example.com', 'Wakad', 'Pune', 'Maharashtra', 'DUMMYA018', 'DUMPA018', '2024-09-15', 'Active'),	('Sameer Khan', '1990-04-09', 'Male', '9100000019', 'sameer@example.com', 'Kurla', 'Mumbai', 'Maharashtra', 'DUMMYA019', 'DUMPA019', '2024-10-01', 'Active'),	('Tanya Roy', '1997-07-26', 'Female', '9100000020', 'tanya@example.com', 'Vartak Nagar', 'Thane', 'Maharashtra', 'DUMMYA020', 'DUMPA020', '2024-10-15', 'Active');
--	--	---	--	---	--	---	--	---	--	---	---	--	--	--	--	--	---	--

LOADED DATA — CUSTOMERS (KYC COLUMNS OMITTED FROM DISPLAY)

customer_id	full_name	date_of_birth	gender	phone_number	email	city	registration_date
1	Aarav Mehta	1995-04-12	Male	9100000001	aarav@example.com	Mumbai	2024-01-10
2	Priya Sharma	1997-08-22	Female	9100000002	priya@example.com	Mumbai	2024-01-15
3	Rohan Patil	1993-02-18	Male	9100000003	rohan@example.com	Pune	2024-02-05
4	Sneha Joshi	1998-11-30	Female	9100000004	sneha@example.com	Nashik	2024-02-20
5	Vikas Rao	1990-06-14	Male	9100000005	vikas@example.com	Nagpur	2024-03-02
6	Ananya Shah	1996-09-05	Female	9100000006	ananya@example.com	Mumbai	2024-03-18
7	Karan Desai	1992-12-10	Male	9100000007	karan@example.com	Thane	2024-04-01
8	Neha Kulkarni	1999-03-25	Female	9100000008	neha@example.com	Pune	2024-04-15
9	Rahul Verma	1994-07-19	Male	9100000009	rahul@example.com	Mumbai	2024-05-01
10	Pooja Deshmukh	1997-01-28	Female	9100000010	pooja@example.com	Aurangabad	2024-05-16
11	Aditya Singh	1991-05-16	Male	9100000011	aditya@example.com	Mumbai	2024-06-05
12	Riya Kapoor	1998-10-11	Female	9100000012	riya@example.com	Mumbai	2024-06-20
13	Siddharth Nair	1989-08-07	Male	9100000013	siddharth@example.com	Pune	2024-07-02
14	Isha Malhotra	1996-02-14	Female	9100000014	isha@example.com	Mumbai	2024-07-18
15	Manish Gupta	1988-11-21	Male	9100000015	manish@example.com	Nashik	2024-08-01
16	Kavya Iyer	1995-06-03	Female	9100000016	kavya@example.com	Nagpur	2024-08-15
17	Arjun Rao	1993-09-27	Male	9100000017	arjun@example.com	Mumbai	2024-09-01
18	Meera Patil	1999-12-19	Female	9100000018	meera@example.com	Pune	2024-09-15
19	Sameer Khan	1990-04-09	Male	9100000019	sameer@example.com	Mumbai	2024-10-01
20	Tanya Roy	1997-07-26	Female	9100000020	tanya@example.com	Thane	2024-10-15

Operational Data Load

Steps 19–28: the accounts, ledger entries, transfers, loan book, repayments, cards, ATM estate, saved payees and staff register that make every later query return real output.

The dataset is internally consistent by design. Account balances tally with the accounts each customer holds, every loan has at least one repayment, every ATM transaction points at a real ATM and a real account, and reference numbers run in unbroken sequences ([TXN000001–TXN000030](#) , [TRF000001–TRF000020](#) , [LP000001–LP000020](#) , [ATM000001–ATM000025](#)).

Step 19 — accounts (25 rows)

Twenty-five accounts across four types. Five customers hold a second account: Aarav Mehta and Manish Gupta each add a Fixed Deposit at 7.00%, while Rohan Patil, Ananya Shah and Aditya Singh add a second operating account. Interest rates follow the product: Savings 4.00%, Current 3.50%, Salary 4.50%, Fixed Deposit 7.00%.

```
STEP 19 — INSERT INTO ACCOUNTS

INSERT INTO accounts
(customer_id, branch_id, account_number, account_type, balance, interest_rate, opening_date, status)
VALUES
( 1, 1, 'ACC100001', 'Savings',      85000.00, 4.00, '2024-01-12', 'Active'),
( 2, 2, 'ACC100002', 'Savings',      62000.00, 4.00, '2024-01-18', 'Active'),
( 3, 3, 'ACC100003', 'Current',     145000.00, 3.50, '2024-02-08', 'Active'),
( 4, 4, 'ACC100004', 'Savings',      48000.00, 4.00, '2024-02-22', 'Active'),
( 5, 5, 'ACC100005', 'Salary',       72000.00, 4.50, '2024-03-05', 'Active'),
( 6, 7, 'ACC100006', 'Savings',      91000.00, 4.00, '2024-03-20', 'Active'),
( 7, 6, 'ACC100007', 'Current',     185000.00, 3.50, '2024-04-03', 'Active'),
( 8, 3, 'ACC100008', 'Savings',      56000.00, 4.00, '2024-04-18', 'Active'),
( 9, 1, 'ACC100009', 'Salary',       68000.00, 4.50, '2024-05-03', 'Active'),
(10, 8, 'ACC100010', 'Savings',      43000.00, 4.00, '2024-05-18', 'Active'),
(11, 7, 'ACC100011', 'Current',     210000.00, 3.50, '2024-06-08', 'Active'),
(12, 2, 'ACC100012', 'Savings',      76000.00, 4.00, '2024-06-22', 'Active'),
(13, 3, 'ACC100013', 'Salary',       83000.00, 4.50, '2024-07-05', 'Active'),
(14, 1, 'ACC100014', 'Savings',      51000.00, 4.00, '2024-07-20', 'Active'),
(15, 4, 'ACC100015', 'Current',     132000.00, 3.50, '2024-08-03', 'Active'),
(16, 5, 'ACC100016', 'Savings',      67000.00, 4.00, '2024-08-18', 'Active'),
(17, 7, 'ACC100017', 'Salary',       95000.00, 4.50, '2024-09-03', 'Active'),
(18, 3, 'ACC100018', 'Savings',      59000.00, 4.00, '2024-09-18', 'Active'),
(19, 2, 'ACC100019', 'Current',     175000.00, 3.50, '2024-10-03', 'Active'),
(20, 6, 'ACC100020', 'Savings',      73000.00, 4.00, '2024-10-18', 'Active'),
( 1, 1, 'ACC100021', 'Fixed Deposit', 250000.00, 7.00, '2024-02-15', 'Active'),
( 3, 3, 'ACC100022', 'Savings',      45000.00, 4.00, '2024-05-10', 'Active'),
( 6, 7, 'ACC100023', 'Current',     120000.00, 3.50, '2024-06-10', 'Active'),
(11, 7, 'ACC100024', 'Savings',      88000.00, 4.00, '2024-07-10', 'Active'),
(15, 4, 'ACC100025', 'Fixed Deposit', 300000.00, 7.00, '2024-08-10', 'Active');

SELECT * FROM accounts;
```

Step 20 — transactions (30 rows)

Three months of ledger activity from January to March 2025: fifteen deposits totalling ₹731,000, ten withdrawals totalling ₹73,500 and five transfers totalling ₹92,000.

```
STEP 20 — INSERT INTO TRANSACTIONS

INSERT INTO transactions
(account_id, transaction_type, amount, transaction_date, description, reference_number)
VALUES
( 1, 'Deposit',      25000.00, '2025-01-05 10:15:00', 'Cash Deposit',      'TXN000001'),
( 2, 'Withdrawal',    8000.00, '2025-01-06 12:30:00', 'ATM Withdrawal',    'TXN000002'),
( 3, 'Deposit',      50000.00, '2025-01-08 11:20:00', 'Business Deposit',  'TXN000003'),
( 4, 'Withdrawal',    5000.00, '2025-01-10 15:10:00', 'Cash Withdrawal',   'TXN000004'),
( 5, 'Deposit',      30000.00, '2025-01-12 09:45:00', 'Salary Credit',     'TXN000005'),
( 6, 'Transfer',      12000.00, '2025-01-15 14:20:00', 'Fund Transfer',     'TXN000006'),
( 7, 'Deposit',      45000.00, '2025-01-18 10:30:00', 'Cash Deposit',      'TXN000007'),
( 8, 'Withdrawal',    7000.00, '2025-01-20 16:15:00', 'ATM Withdrawal',    'TXN000008'),
( 9, 'Deposit',      28000.00, '2025-01-22 11:00:00', 'Salary Credit',     'TXN000009'),
(10, 'Withdrawal',    6000.00, '2025-01-25 13:40:00', 'Cash Withdrawal',   'TXN000010'),
(11, 'Deposit',      60000.00, '2025-02-02 10:10:00', 'Business Deposit',  'TXN000011'),
(12, 'Transfer',      15000.00, '2025-02-05 12:25:00', 'Fund Transfer',     'TXN000012'),
(13, 'Deposit',      35000.00, '2025-02-08 09:30:00', 'Salary Credit',     'TXN000013'),
(14, 'Withdrawal',    9000.00, '2025-02-10 15:45:00', 'ATM Withdrawal',    'TXN000014'),
(15, 'Deposit',      40000.00, '2025-02-12 11:50:00', 'Cash Deposit',      'TXN000015'),
(16, 'Withdrawal',    5000.00, '2025-02-15 14:10:00', 'ATM Withdrawal',    'TXN000016'),
(17, 'Deposit',      55000.00, '2025-02-18 10:45:00', 'Salary Credit',     'TXN000017'),
(18, 'Transfer',      18000.00, '2025-02-20 13:15:00', 'Fund Transfer',     'TXN000018'),
(19, 'Deposit',      48000.00, '2025-02-22 11:35:00', 'Business Deposit',  'TXN000019'),
(20, 'Withdrawal',    7500.00, '2025-02-25 16:20:00', 'Cash Withdrawal',   'TXN000020'),
(21, 'Deposit',      100000.00, '2025-03-01 10:05:00', 'Fixed Deposit Credit', 'TXN000021'),
(22, 'Withdrawal',    4000.00, '2025-03-03 12:40:00', 'Cash Withdrawal',   'TXN000022'),
(23, 'Deposit',      30000.00, '2025-03-05 09:55:00', 'Business Deposit',  'TXN000023'),
(24, 'Transfer',      22000.00, '2025-03-08 14:30:00', 'Fund Transfer',     'TXN000024'),
(25, 'Deposit',      120000.00, '2025-03-10 10:25:00', 'Fixed Deposit Credit', 'TXN000025'),
( 1, 'Withdrawal',    10000.00, '2025-03-12 15:10:00', 'ATM Withdrawal',    'TXN000026'),
( 3, 'Transfer',      25000.00, '2025-03-15 11:45:00', 'Fund Transfer',     'TXN000027'),
( 6, 'Deposit',      20000.00, '2025-03-18 13:20:00', 'Cash Deposit',      'TXN000028'),
(11, 'Withdrawal',    12000.00, '2025-03-20 16:00:00', 'Cash Withdrawal',    'TXN000029'),
(15, 'Deposit',      45000.00, '2025-03-22 10:50:00', 'Cash Deposit',      'TXN000030');
```

Step 21 — fund_transfers (20 rows)

A chain of twenty transfers in which each account sends to the next and account 20 closes the loop back to account 1. All four Indian payment rails are represented: UPI, NEFT, RTGS and IMPS.

STEP 21 — INSERT INTO FUND_TRANSFERS

```
INSERT INTO fund_transfers
(sender_account_id, receiver_account_id, amount, transfer_date,
 transfer_mode, transfer_status, reference_number)
VALUES
( 1, 2, 5000.00, '2025-01-07 10:30:00', 'UPI', 'Completed', 'TRF000001'),
( 2, 3, 12000.00, '2025-01-12 11:15:00', 'NEFT', 'Completed', 'TRF000002'),
( 3, 4, 25000.00, '2025-01-18 14:20:00', 'RTGS', 'Completed', 'TRF000003'),
( 4, 5, 7000.00, '2025-01-22 09:45:00', 'UPI', 'Completed', 'TRF000004'),
( 5, 6, 15000.00, '2025-01-28 16:10:00', 'IMPS', 'Completed', 'TRF000005'),
( 6, 7, 18000.00, '2025-02-03 10:20:00', 'NEFT', 'Completed', 'TRF000006'),
( 7, 8, 22000.00, '2025-02-08 13:30:00', 'RTGS', 'Completed', 'TRF000007'),
( 8, 9, 6500.00, '2025-02-12 15:40:00', 'UPI', 'Completed', 'TRF000008'),
( 9, 10, 9500.00, '2025-02-16 11:50:00', 'IMPS', 'Completed', 'TRF000009'),
(10, 11, 14000.00, '2025-02-20 12:15:00', 'NEFT', 'Completed', 'TRF000010'),
(11, 12, 30000.00, '2025-02-25 14:45:00', 'RTGS', 'Completed', 'TRF000011'),
(12, 13, 8500.00, '2025-03-01 10:35:00', 'UPI', 'Completed', 'TRF000012'),
(13, 14, 16000.00, '2025-03-05 13:20:00', 'IMPS', 'Completed', 'TRF000013'),
(14, 15, 11000.00, '2025-03-10 15:10:00', 'NEFT', 'Completed', 'TRF000014'),
(15, 16, 20000.00, '2025-03-15 11:25:00', 'RTGS', 'Completed', 'TRF000015'),
(16, 17, 7500.00, '2025-03-20 09:55:00', 'UPI', 'Completed', 'TRF000016'),
(17, 18, 13500.00, '2025-03-25 14:30:00', 'IMPS', 'Completed', 'TRF000017'),
(18, 19, 24000.00, '2025-04-02 12:10:00', 'NEFT', 'Completed', 'TRF000018'),
(19, 20, 17500.00, '2025-04-07 16:20:00', 'UPI', 'Completed', 'TRF000019'),
(20, 1, 9000.00, '2025-04-12 10:40:00', 'IMPS', 'Completed', 'TRF000020');
```

SCRIPT CORRECTION APPLIED

The original script repeated the `CREATE TABLE fund_transfers` and this entire insert as Steps 21A and 21B. Both repetitions are removed here — running them produces `ERROR 1050` followed by twenty duplicate-key errors. Section 09 documents the defect and the corrected script in full.

Step 22 — loans (20 rows)

One loan per customer across five products. Home loans dominate the book at ₹13.7 million over five accounts, with tenures running to 240 months. Rates are risk-priced: education loans from 7.75%, home loans around 8.30–8.50%, personal loans up to 11.50%.

STEP 22 — INSERT INTO LOANS

```
DESCRIBE loans;

INSERT INTO loans
(customer_id, branch_id, loan_type, loan_amount, interest_rate,
 tenure_months, start_date, end_date, loan_status)
VALUES
( 1, 1, 'Home Loan', 2500000.00, 8.50, 240, '2024-01-15', '2044-01-15', 'Active'),
( 2, 2, 'Personal Loan', 500000.00, 11.50, 60, '2024-02-10', '2029-02-10', 'Active'),
( 3, 3, 'Business Loan', 1500000.00, 10.25, 120, '2024-02-20', '2034-02-20', 'Active'),
( 4, 4, 'Car Loan', 800000.00, 9.00, 84, '2024-03-05', '2031-03-05', 'Active'),
( 5, 5, 'Education Loan', 600000.00, 8.00, 96, '2024-03-18', '2032-03-18', 'Active'),
( 6, 7, 'Home Loan', 3200000.00, 8.25, 240, '2024-04-01', '2044-04-01', 'Active'),
( 7, 6, 'Personal Loan', 750000.00, 11.00, 60, '2024-04-15', '2029-04-15', 'Active'),
( 8, 3, 'Car Loan', 950000.00, 8.75, 84, '2024-05-02', '2031-05-02', 'Active'),
( 9, 1, 'Business Loan', 2000000.00, 10.00, 120, '2024-05-20', '2034-05-20', 'Active'),
(10, 8, 'Education Loan', 450000.00, 7.75, 72, '2024-06-10', '2030-06-10', 'Active'),
(11, 7, 'Home Loan', 2800000.00, 8.40, 240, '2024-06-25', '2044-06-25', 'Active'),
(12, 2, 'Personal Loan', 400000.00, 11.25, 48, '2024-07-08', '2028-07-08', 'Active'),
(13, 3, 'Business Loan', 1750000.00, 10.50, 120, '2024-07-22', '2034-07-22', 'Active'),
(14, 1, 'Car Loan', 700000.00, 9.10, 84, '2024-08-05', '2031-08-05', 'Active'),
(15, 4, 'Home Loan', 2200000.00, 8.30, 180, '2024-08-20', '2039-08-20', 'Active'),
(16, 5, 'Personal Loan', 550000.00, 11.00, 60, '2024-09-10', '2029-09-10', 'Active'),
(17, 7, 'Business Loan', 1250000.00, 10.25, 120, '2024-09-25', '2034-09-25', 'Active'),
(18, 3, 'Education Loan', 500000.00, 7.90, 84, '2024-10-05', '2031-10-05', 'Active'),
(19, 2, 'Car Loan', 900000.00, 8.90, 84, '2024-10-20', '2031-10-20', 'Active'),
(20, 6, 'Home Loan', 3000000.00, 8.45, 240, '2024-11-05', '2044-11-05', 'Active');
```

Step 23 — loan_payments (20 rows)

One instalment recorded per loan, paid by bank transfer, UPI or auto debit. Together they account for ₹358,500 against a book of ₹27.3 million, which is what the remaining-balance view in Section 06 reports.

STEP 23 — INSERT INTO LOAN_PAYMENTS

```
DESCRIBE loan_payments;

INSERT INTO loan_payments
(loan_id, payment_date, payment_amount, payment_method, payment_status, reference_number)
VALUES
( 1, '2025-01-10', 25000.00, 'Bank Transfer', 'Paid', 'LP000001'),
( 2, '2025-01-12', 11000.00, 'UPI', 'Paid', 'LP000002'),
( 3, '2025-01-15', 22000.00, 'Bank Transfer', 'Paid', 'LP000003'),
( 4, '2025-01-18', 14500.00, 'Auto Debit', 'Paid', 'LP000004'),
( 5, '2025-01-20', 8500.00, 'UPI', 'Paid', 'LP000005'),
( 6, '2025-01-25', 28000.00, 'Bank Transfer', 'Paid', 'LP000006'),
( 7, '2025-02-02', 16500.00, 'Auto Debit', 'Paid', 'LP000007'),
( 8, '2025-02-05', 15500.00, 'UPI', 'Paid', 'LP000008'),
( 9, '2025-02-08', 26500.00, 'Bank Transfer', 'Paid', 'LP000009'),
(10, '2025-02-12', 7500.00, 'UPI', 'Paid', 'LP000010'),
(11, '2025-02-15', 26000.00, 'Auto Debit', 'Paid', 'LP000011'),
(12, '2025-02-18', 12000.00, 'Bank Transfer', 'Paid', 'LP000012'),
(13, '2025-02-22', 24000.00, 'UPI', 'Paid', 'LP000013'),
(14, '2025-02-25', 13500.00, 'Auto Debit', 'Paid', 'LP000014'),
(15, '2025-03-02', 23500.00, 'Bank Transfer', 'Paid', 'LP000015'),
(16, '2025-03-05', 12500.00, 'UPI', 'Paid', 'LP000016');
```

```
(17, '2025-03-08', 21000.00, 'Bank Transfer', 'Paid', 'LP000017'),
(18, '2025-03-12', 9000.00, 'Auto Debit', 'Paid', 'LP000018'),
(19, '2025-03-15', 15000.00, 'UPI', 'Paid', 'LP000019'),
(20, '2025-03-18', 27000.00, 'Bank Transfer', 'Paid', 'LP000020');
```

```
SELECT * FROM loan_payments;
```

Step 24 — cards (20 rows)

Fourteen debit cards and six credit cards, each issued against the customer’s primary account with a five-year validity.

STEP 24 — INSERT INTO CARDS

```
DESCRIBE cards;

INSERT INTO cards
(customer_id, account_id, card_type, card_number_masked, issue_date, expiry_date, card_status)
VALUES
( 1, 1, 'Debit', 'XXXX-XXXX-XXXX-1001', '2024-01-20', '2029-01-20', 'Active'),
( 2, 2, 'Debit', 'XXXX-XXXX-XXXX-1002', '2024-01-25', '2029-01-25', 'Active'),
( 3, 3, 'Credit', 'XXXX-XXXX-XXXX-1003', '2024-02-15', '2029-02-15', 'Active'),
( 4, 4, 'Debit', 'XXXX-XXXX-XXXX-1004', '2024-02-28', '2029-02-28', 'Active'),
( 5, 5, 'Debit', 'XXXX-XXXX-XXXX-1005', '2024-03-10', '2029-03-10', 'Active'),
( 6, 6, 'Credit', 'XXXX-XXXX-XXXX-1006', '2024-03-25', '2029-03-25', 'Active'),
( 7, 7, 'Debit', 'XXXX-XXXX-XXXX-1007', '2024-04-10', '2029-04-10', 'Active'),
( 8, 8, 'Debit', 'XXXX-XXXX-XXXX-1008', '2024-04-25', '2029-04-25', 'Active'),
( 9, 9, 'Credit', 'XXXX-XXXX-XXXX-1009', '2024-05-10', '2029-05-10', 'Active'),
(10, 10, 'Debit', 'XXXX-XXXX-XXXX-1010', '2024-05-25', '2029-05-25', 'Active'),
(11, 11, 'Debit', 'XXXX-XXXX-XXXX-1011', '2024-06-15', '2029-06-15', 'Active'),
(12, 12, 'Credit', 'XXXX-XXXX-XXXX-1012', '2024-06-28', '2029-06-28', 'Active'),
(13, 13, 'Debit', 'XXXX-XXXX-XXXX-1013', '2024-07-12', '2029-07-12', 'Active'),
(14, 14, 'Debit', 'XXXX-XXXX-XXXX-1014', '2024-07-28', '2029-07-28', 'Active'),
(15, 15, 'Credit', 'XXXX-XXXX-XXXX-1015', '2024-08-12', '2029-08-12', 'Active'),
(16, 16, 'Debit', 'XXXX-XXXX-XXXX-1016', '2024-08-28', '2029-08-28', 'Active'),
(17, 17, 'Debit', 'XXXX-XXXX-XXXX-1017', '2024-09-12', '2029-09-12', 'Active'),
(18, 18, 'Credit', 'XXXX-XXXX-XXXX-1018', '2024-09-28', '2029-09-28', 'Active'),
(19, 19, 'Debit', 'XXXX-XXXX-XXXX-1019', '2024-10-12', '2029-10-12', 'Active'),
(20, 20, 'Debit', 'XXXX-XXXX-XXXX-1020', '2024-10-28', '2029-10-28', 'Active');

SELECT * FROM cards;
```

Step 25 — atms (10 rows)

Ten machines across seven branches. Mumbai Central, Andheri and Pune Main run two ATMs each, which is what makes the ATM activity roll-up in Section 07 meaningful.

STEP 25 — INSERT INTO ATMS

```
DESCRIBE atms;

INSERT INTO atms
(branch_id, atm_location, city, installation_date, status)
VALUES
(1, 'Mumbai Central Railway Road', 'Mumbai', '2022-01-15', 'Active'),
(1, 'MG Road Shopping Complex', 'Mumbai', '2022-06-20', 'Active'),
(2, 'Andheri West Metro Station', 'Mumbai', '2022-03-10', 'Active'),
(2, 'Andheri Market Area', 'Mumbai', '2023-01-25', 'Active'),
(3, 'Kothrud Main Road', 'Pune', '2022-08-12', 'Active'),
(3, 'Pune Camp Area', 'Pune', '2023-04-18', 'Active'),
(4, 'College Road', 'Nashik', '2022-11-05', 'Active'),
(5, 'Civil Lines', 'Nagpur', '2023-02-14', 'Active'),
(6, 'Thane West Station Road', 'Thane', '2023-05-20', 'Active'),
(7, 'Bandra Linking Road', 'Mumbai', '2023-07-10', 'Active');

SELECT * FROM atms;
```

LOADED DATA — ATMS

atm_id	branch_id	atm_location	city	installation_date	status
1	1	Mumbai Central Railway Road	Mumbai	2022-01-15	Active
2	1	MG Road Shopping Complex	Mumbai	2022-06-20	Active
3	2	Andheri West Metro Station	Mumbai	2022-03-10	Active
4	2	Andheri Market Area	Mumbai	2023-01-25	Active
5	3	Kothrud Main Road	Pune	2022-08-12	Active
6	3	Pune Camp Area	Pune	2023-04-18	Active
7	4	College Road	Nashik	2022-11-05	Active
8	5	Civil Lines	Nagpur	2023-02-14	Active
9	6	Thane West Station Road	Thane	2023-05-20	Active
10	7	Bandra Linking Road	Mumbai	2023-07-10	Active

Step 26 — atm_transactions (25 rows)

Fourteen withdrawals, seven deposits and four balance enquiries. The enquiries carry an amount of zero, which is why the ATM analysis in Section 07 shows a zero total for that category rather than excluding it.

STEP 26 — INSERT INTO ATM_TRANSACTIONS

```
DESCRIBE atm_transactions;

INSERT INTO atm_transactions
```

```
(atm_id, account_id, transaction_type, amount, transaction_date, status, reference_number)
VALUES
( 1, 1, 'Withdrawal',      5000.00, '2025-01-05 10:15:00', 'Successful', 'ATM000001'),
( 2, 2, 'Withdrawal',      8000.00, '2025-01-08 12:30:00', 'Successful', 'ATM000002'),
( 3, 3, 'Deposit',         15000.00, '2025-01-12 11:20:00', 'Successful', 'ATM000003'),
( 4, 4, 'Withdrawal',      4000.00, '2025-01-15 15:10:00', 'Successful', 'ATM000004'),
( 5, 5, 'Balance Inquiry',    0.00, '2025-01-18 09:45:00', 'Successful', 'ATM000005'),
( 6, 6, 'Withdrawal',      10000.00, '2025-01-20 14:20:00', 'Successful', 'ATM000006'),
( 7, 7, 'Deposit',         20000.00, '2025-01-25 10:30:00', 'Successful', 'ATM000007'),
( 8, 8, 'Withdrawal',      6000.00, '2025-02-02 16:15:00', 'Successful', 'ATM000008'),
( 9, 9, 'Withdrawal',      7500.00, '2025-02-05 11:00:00', 'Successful', 'ATM000009'),
(10,10, 'Balance Inquiry',    0.00, '2025-02-08 13:40:00', 'Successful', 'ATM000010'),
( 1,11, 'Withdrawal',      12000.00, '2025-02-12 10:10:00', 'Successful', 'ATM000011'),
( 2,12, 'Deposit',         18000.00, '2025-02-15 12:25:00', 'Successful', 'ATM000012'),
( 3,13, 'Withdrawal',      9000.00, '2025-02-18 14:30:00', 'Successful', 'ATM000013'),
( 4,14, 'Balance Inquiry',    0.00, '2025-02-20 15:45:00', 'Successful', 'ATM000014'),
( 5,15, 'Withdrawal',      6500.00, '2025-02-25 11:50:00', 'Successful', 'ATM000015'),
( 6,16, 'Deposit',         25000.00, '2025-03-01 09:30:00', 'Successful', 'ATM000016'),
( 7,17, 'Withdrawal',      11000.00, '2025-03-05 13:15:00', 'Successful', 'ATM000017'),
( 8,18, 'Withdrawal',      5500.00, '2025-03-08 16:20:00', 'Successful', 'ATM000018'),
( 9,19, 'Deposit',         22000.00, '2025-03-12 10:40:00', 'Successful', 'ATM000019'),
(10,20, 'Balance Inquiry',    0.00, '2025-03-15 12:10:00', 'Successful', 'ATM000020'),
( 1,21, 'Withdrawal',      15000.00, '2025-03-18 14:25:00', 'Successful', 'ATM000021'),
( 3,22, 'Deposit',         17000.00, '2025-03-20 11:35:00', 'Successful', 'ATM000022'),
( 5,23, 'Withdrawal',      7000.00, '2025-03-22 15:00:00', 'Successful', 'ATM000023'),
( 7,24, 'Withdrawal',      8500.00, '2025-03-25 10:50:00', 'Successful', 'ATM000024'),
( 9,25, 'Deposit',         30000.00, '2025-03-28 13:20:00', 'Successful', 'ATM000025');
```

Step 27 — beneficiaries (20 rows)

One saved payee per customer, spread across four external banks and tagged with the relationship — brother, sister, mother, father or friend.

STEP 27 — INSERT INTO BENEFICIARIES

```
DESCRIBE beneficiaries;

INSERT INTO beneficiaries
(customer_id, beneficiary_name, beneficiary_account_number, beneficiary_bank_name,
 ifsc_code, relationship, added_date, status)
VALUES
( 1, 'Rahul Mehta',      'BENACC10001', 'ABC Bank',      'ABCB0001001', 'Brother', '2025-01-05', 'Active'),
( 2, 'Neha Sharma',     'BENACC10002', 'XYZ Bank',      'XYZB0001002', 'Sister',  '2025-01-08', 'Active'),
( 3, 'Amit Patil',      'BENACC10003', 'National Bank', 'NATB0001003', 'Friend',  '2025-01-12', 'Active'),
( 4, 'Sneha Joshi',     'BENACC10004', 'City Bank',     'CITY0001004', 'Sister',  '2025-01-15', 'Active'),
( 5, 'Vikas Rao',       'BENACC10005', 'ABC Bank',      'ABCB0001005', 'Father',  '2025-01-18', 'Active'),
( 6, 'Ananya Shah',     'BENACC10006', 'XYZ Bank',      'XYZB0001006', 'Friend',  '2025-01-20', 'Active'),
( 7, 'Karan Desai',     'BENACC10007', 'National Bank', 'NATB0001007', 'Brother', '2025-01-25', 'Active'),
( 8, 'Pooja Kulkarni',  'BENACC10008', 'City Bank',     'CITY0001008', 'Mother',  '2025-02-02', 'Active'),
( 9, 'Riya Verma',      'BENACC10009', 'ABC Bank',      'ABCB0001009', 'Friend',  '2025-02-05', 'Active'),
(10, 'Aditya Singh',    'BENACC10010', 'XYZ Bank',      'XYZB0001010', 'Brother', '2025-02-08', 'Active'),
(11, 'Meera Kapoor',   'BENACC10011', 'National Bank', 'NATB0001011', 'Sister',  '2025-02-12', 'Active'),
(12, 'Siddharth Nair',  'BENACC10012', 'City Bank',     'CITY0001012', 'Friend',  '2025-02-15', 'Active'),
(13, 'Isha Malhotra',   'BENACC10013', 'ABC Bank',      'ABCB0001013', 'Mother',  '2025-02-18', 'Active'),
(14, 'Arjun Rao',       'BENACC10014', 'XYZ Bank',      'XYZB0001014', 'Father',  '2025-02-20', 'Active'),
(15, 'Kavya Iyer',      'BENACC10015', 'National Bank', 'NATB0001015', 'Sister',  '2025-02-25', 'Active'),
(16, 'Manish Gupta',    'BENACC10016', 'City Bank',     'CITY0001016', 'Friend',  '2025-03-01', 'Active'),
(17, 'Meera Patil',     'BENACC10017', 'ABC Bank',      'ABCB0001017', 'Mother',  '2025-03-05', 'Active'),
(18, 'Sameer Khan',     'BENACC10018', 'XYZ Bank',      'XYZB0001018', 'Brother', '2025-03-08', 'Active'),
(19, 'Tanya Roy',       'BENACC10019', 'National Bank', 'NATB0001019', 'Friend',  '2025-03-12', 'Active'),
(20, 'Aarav Shah',     'BENACC10020', 'City Bank',     'CITY0001020', 'Father',  '2025-03-15', 'Active');
```

```
SELECT * FROM beneficiaries;
```

Step 28 — employees (20 rows)

Between two and three staff per branch, each mapped to a department by `department_id`. Salaries range from ₹38,000 for a customer executive to ₹90,000 for a branch manager.

STEP 28 — INSERT INTO EMPLOYEES

```
DESCRIBE employees;

INSERT INTO employees
(branch_id, employee_name, designation, department, phone_number, email,
 joining_date, salary, status, department_id)
VALUES
(1, 'Rahul Sharma',      'Branch Manager',      'Management',      '9876500001', 'rahul.sharma@bank.com', '2021-04-12', 85000.00, 'Active', 1),
(1, 'Priya Mehta',      'Relationship Manager', 'Customer Service', '9876500002', 'priya.mehta@bank.com', '2022-06-15', 62000.00, 'Active', 2),
(2, 'Amit Patil',       'Assistant Manager',   'Loans',            '9876500003', 'amit.patil@bank.com', '2021-08-20', 70000.00, 'Active', 3),
(2, 'Neha Shah',       'Loan Officer',        'Loans',            '9876500004', 'neha.shah@bank.com', '2023-01-10', 55000.00, 'Active', 3),
(3, 'Vikas Rao',       'Cashier',             'Operations',        '9876500005', 'vikas.rao@bank.com', '2022-03-18', 42000.00, 'Active', 4),
(3, 'Sneha Joshi',     'Accountant',          'Finance',           '9876500006', 'sneha.joshi@bank.com', '2021-11-05', 58000.00, 'Active', 5),
(4, 'Karan Desai',     'Branch Manager',      'Management',      '9876500007', 'karan.desai@bank.com', '2020-07-15', 90000.00, 'Active', 1),
(4, 'Pooja Kulkarni',   'Customer Executive',  'Customer Service', '9876500008', 'pooja.kulkarni@bank.com', '2023-02-20', 38000.00, 'Active', 2),
(5, 'Aditya Singh',    'Loan Officer',        'Loans',            '9876500009', 'aditya.singh@bank.com', '2022-09-12', 52000.00, 'Active', 3),
(5, 'Riya Verma',      'Cashier',             'Operations',        '9876500010', 'riya.verma@bank.com', '2023-04-10', 40000.00, 'Active', 4),
(6, 'Sameer Khan',     'IT Officer',          'IT',               '9876500011', 'sameer.khan@bank.com', '2021-05-22', 68000.00, 'Active', 6),
(6, 'Meera Kapoor',    'HR Executive',        'Human Resources',  '9876500012', 'meera.kapoor@bank.com', '2022-10-15', 48000.00, 'Active', 7),
(7, 'Arjun Nair',      'Branch Manager',      'Management',      '9876500013', 'arjun.nair@bank.com', '2020-12-01', 88000.00, 'Active', 1),
(7, 'Isha Malhotra',   'Security Officer',    'Security',         '9876500014', 'isha.malhotra@bank.com', '2023-06-18', 45000.00, 'Active', 8),
(8, 'Manish Gupta',    'Accountant',          'Finance',           '9876500015', 'manish.gupta@bank.com', '2021-09-08', 60000.00, 'Active', 5),
(8, 'Tanya Roy',       'Customer Executive',  'Customer Service', '9876500016', 'tanya.roy@bank.com', '2022-12-10', 39000.00, 'Active', 2),
(1, 'Siddharth Nair',  'IT Officer',          'IT',               '9876500017', 'siddharth.nair@bank.com', '2023-03-15', 65000.00, 'Active', 6),
(2, 'Kavya Iyer',      'HR Executive',        'Human Resources',  '9876500018', 'kavya.iyer@bank.com', '2023-07-05', 47000.00, 'Active', 7),
(3, 'Manoj Joshi',     'Security Officer',    'Security',         '9876500019', 'manoj.joshi@bank.com', '2022-05-12', 44000.00, 'Active', 8),
(4, 'Ananya Shah',    'Operations Officer',  'Operations',        '9876500020', 'ananya.shah@bank.com', '2024-01-08', 46000.00, 'Active', 4);
```

```
SELECT * FROM employees;
```

Load verification

Row counts are checked immediately after loading, which is the simplest guard against a partially applied script.

VERIFICATION COUNTS

```
SELECT COUNT(*) AS total_atms      FROM atms;
SELECT COUNT(*) AS total_beneficiaries FROM beneficiaries;
SELECT COUNT(*) AS total_employees  FROM employees;
```

VERIFICATION OUTPUT

Check	Expected	Returned
total_atms	10	10
total_beneficiaries	20	20
total_employees	20	20

Analytical Views

Steps 29–38: ten views that flatten the schema's joins into readable result sets, each shown with the output it returns against the seeded data.

Views are where the schema stops being a set of tables and starts being a reporting tool. Each one below hides a two-, three- or four-table join behind a single name, so a later query can ask “*which customers hold the largest balances*” without restating the join every time. Every result set in this section is the full output, not a sample.

Step 29 — customer_account_summary

Joins each account to the customer who owns it. Customers holding more than one account appear once per account, which is exactly what an account-level report needs.

STEP 29 — CREATE VIEW CUSTOMER_ACCOUNT_SUMMARY

CREATE VIEW customer_account_summary AS
SELECT
 c.customer_id,
 c.full_name,
 a.account_id,
 a.account_number,
 a.account_type,
 a.balance,
 a.interest_rate,
 a.status AS account_status
FROM customers c
JOIN accounts a
 ON c.customer_id = a.customer_id;

SELECT * FROM customer_account_summary;

OUTPUT — 25 ROWS

customer_id		full_name	account_id	account_number	account_type	balance	interest_rate	account_status	
	1	Aarav Mehta		1	ACC100001	Savings	85000	4.00	Active
	1	Aarav Mehta		21	ACC100021	Fixed Deposit	250000	7.00	Active
	2	Priya Sharma		2	ACC100002	Savings	62000	4.00	Active
	3	Rohan Patil		3	ACC100003	Current	145000	3.50	Active
	3	Rohan Patil		22	ACC100022	Savings	45000	4.00	Active
	4	Sneha Joshi		4	ACC100004	Savings	48000	4.00	Active
	5	Vikas Rao		5	ACC100005	Salary	72000	4.50	Active
	6	Ananya Shah		6	ACC100006	Savings	91000	4.00	Active
	6	Ananya Shah		23	ACC100023	Current	120000	3.50	Active
	7	Karan Desai		7	ACC100007	Current	185000	3.50	Active
	8	Neha Kulkarni		8	ACC100008	Savings	56000	4.00	Active
	9	Rahul Verma		9	ACC100009	Salary	68000	4.50	Active
	10	Pooja Deshmukh		10	ACC100010	Savings	43000	4.00	Active
	11	Aditya Singh		11	ACC100011	Current	210000	3.50	Active
	11	Aditya Singh		24	ACC100024	Savings	88000	4.00	Active
	12	Riya Kapoor		12	ACC100012	Savings	76000	4.00	Active
	13	Siddharth Nair		13	ACC100013	Salary	83000	4.50	Active
	14	Isha Malhotra		14	ACC100014	Savings	51000	4.00	Active
	15	Manish Gupta		15	ACC100015	Current	132000	3.50	Active
	15	Manish Gupta		25	ACC100025	Fixed Deposit	300000	7.00	Active
	16	Kavya Iyer		16	ACC100016	Savings	67000	4.00	Active
	17	Arjun Rao		17	ACC100017	Salary	95000	4.50	Active
	18	Meera Patil		18	ACC100018	Savings	59000	4.00	Active
	19	Sameer Khan		19	ACC100019	Current	175000	3.50	Active
	20	Tanya Roy		20	ACC100020	Savings	73000	4.00	Active

Step 30 — loan_summary

Attaches the borrower’s name to every loan, giving a credit officer a single readable view of the loan book.

STEP 30 — CREATE VIEW LOAN_SUMMARY

CREATE VIEW loan_summary AS
SELECT
 l.loan_id,
 c.customer_id,
 c.full_name,
 l.loan_type,
 l.loan_amount,
 l.interest_rate,
 l.tenure_months,
 l.start_date,
 l.end_date,
 l.loan_status
FROM loans l
JOIN customers c
 ON l.customer_id = c.customer_id;

```
SELECT * FROM loan_summary;
```

OUTPUT — 20 ROWS

loan_id	customer_id	full_name	loan_type	loan_amount	interest_rate	tenure_months	start_date	end_date	loan_status
1	1	Aarav Mehta	Home Loan	2500000	8.50	240	2024-01-15	2044-01-15	Active
2	2	Priya Sharma	Personal Loan	500000	11.50	60	2024-02-10	2029-02-10	Active
3	3	Rohan Patil	Business Loan	1500000	10.25	120	2024-02-20	2034-02-20	Active
4	4	Sneha Joshi	Car Loan	800000	9.00	84	2024-03-05	2031-03-05	Active
5	5	Vikas Rao	Education Loan	600000	8.00	96	2024-03-18	2032-03-18	Active
6	6	Ananya Shah	Home Loan	3200000	8.25	240	2024-04-01	2044-04-01	Active
7	7	Karan Desai	Personal Loan	750000	11.00	60	2024-04-15	2029-04-15	Active
8	8	Neha Kulkarni	Car Loan	950000	8.75	84	2024-05-02	2031-05-02	Active
9	9	Rahul Verma	Business Loan	2000000	10.00	120	2024-05-20	2034-05-20	Active
10	10	Pooja Deshmukh	Education Loan	450000	7.75	72	2024-06-10	2030-06-10	Active
11	11	Aditya Singh	Home Loan	2800000	8.40	240	2024-06-25	2044-06-25	Active
12	12	Riya Kapoor	Personal Loan	400000	11.25	48	2024-07-08	2028-07-08	Active
13	13	Siddharth Nair	Business Loan	1750000	10.50	120	2024-07-22	2034-07-22	Active
14	14	Isha Malhotra	Car Loan	700000	9.10	84	2024-08-05	2031-08-05	Active
15	15	Manish Gupta	Home Loan	2200000	8.30	180	2024-08-20	2039-08-20	Active
16	16	Kavya Iyer	Personal Loan	550000	11.00	60	2024-09-10	2029-09-10	Active
17	17	Arjun Rao	Business Loan	1250000	10.25	120	2024-09-25	2034-09-25	Active
18	18	Meera Patil	Education Loan	500000	7.90	84	2024-10-05	2031-10-05	Active
19	19	Sameer Khan	Car Loan	900000	8.90	84	2024-10-20	2031-10-20	Active
20	20	Tanya Roy	Home Loan	3000000	8.45	240	2024-11-05	2044-11-05	Active

Step 31 — transaction_summary

A three-table join — transaction to account to customer — so that every ledger line carries both the account number it hit and the name behind it.

```
STEP 31 — CREATE VIEW TRANSACTION_SUMMARY

DESCRIBE transactions;

CREATE VIEW transaction_summary AS
SELECT
    t.transaction_id,
    c.customer_id,
    c.full_name,
    a.account_id,
    a.account_number,
    a.account_type,
    t.transaction_type,
    t.amount,
    t.transaction_date,
    t.description,
    t.reference_number
FROM transactions t
JOIN accounts a
    ON t.account_id = a.account_id
JOIN customers c
    ON a.customer_id = c.customer_id;

SELECT * FROM transaction_summary;
```

OUTPUT — 30 ROWS

transaction_id	customer_id	full_name	account_id	account_number	account_type	transaction_type	amount	transaction_date	description	reference_number
1	1	Aarav Mehta	1	ACC100001	Savings	Deposit	25000	2025-01-05 10:15	Cash Deposit	TXN000001
26	1	Aarav Mehta	1	ACC100001	Savings	Withdrawal	10000	2025-03-12 15:10	ATM Withdrawal	TXN000026
21	1	Aarav Mehta	21	ACC100021	Fixed Deposit	Deposit	100000	2025-03-01 10:05	Fixed Deposit Credit	TXN000021
2	2	Priya Sharma	2	ACC100002	Savings	Withdrawal	8000	2025-01-06 12:30	ATM Withdrawal	TXN000002
3	3	Rohan Patil	3	ACC100003	Current	Deposit	50000	2025-01-08 11:20	Business Deposit	TXN000003
27	3	Rohan Patil	3	ACC100003	Current	Transfer	25000	2025-03-15 11:45	Fund Transfer	TXN000027
22	3	Rohan Patil	22	ACC100022	Savings	Withdrawal	4000	2025-03-03 12:40	Cash Withdrawal	TXN000022
4	4	Sneha Joshi	4	ACC100004	Savings	Withdrawal	5000	2025-01-10 15:10	Cash Withdrawal	TXN000004
5	5	Vikas Rao	5	ACC100005	Salary	Deposit	30000	2025-01-12 09:45	Salary Credit	TXN000005
6	6	Ananya Shah	6	ACC100006	Savings	Transfer	12000	2025-01-15 14:20	Fund Transfer	TXN000006
28	6	Ananya Shah	6	ACC100006	Savings	Deposit	20000	2025-03-18 13:20	Cash Deposit	TXN000028
23	6	Ananya Shah	23	ACC100023	Current	Deposit	30000	2025-03-05 09:55	Business Deposit	TXN000023
7	7	Karan Desai	7	ACC100007	Current	Deposit	45000	2025-01-18 10:30	Cash Deposit	TXN000007
8	8	Neha Kulkarni	8	ACC100008	Savings	Withdrawal	7000	2025-01-20 16:15	ATM Withdrawal	TXN000008
9	9	Rahul Verma	9	ACC100009	Salary	Deposit	28000	2025-01-22 11:00	Salary Credit	TXN000009
10	10	Pooja Deshmukh	10	ACC100010	Savings	Withdrawal	6000	2025-01-25 13:40	Cash Withdrawal	TXN000010
11	11	Aditya Singh	11	ACC100011	Current	Deposit	60000	2025-02-02 10:10	Business Deposit	TXN000011
29	11	Aditya Singh	11	ACC100011	Current	Withdrawal	12000	2025-03-20 16:00	Cash Withdrawal	TXN000029
24	11	Aditya Singh	24	ACC100024	Savings	Transfer	22000	2025-03-08 14:30	Fund Transfer	TXN000024
12	12	Riya Kapoor	12	ACC100012	Savings	Transfer	15000	2025-02-05 12:25	Fund Transfer	TXN000012
13	13	Siddharth Nair	13	ACC100013	Salary	Deposit	35000	2025-02-08 09:30	Salary Credit	TXN000013

transaction_id	customer_id	full_name	account_id	account_number	account_type	transaction_type	amount	transaction_date	description	reference_number
14	14	Isha Malhotra	14	ACC100014	Savings	Withdrawal	9000	2025-02-10 15:45	ATM Withdrawal	TXN000014
15	15	Manish Gupta	15	ACC100015	Current	Deposit	40000	2025-02-12 11:50	Cash Deposit	TXN000015
30	15	Manish Gupta	15	ACC100015	Current	Deposit	45000	2025-03-22 10:50	Cash Deposit	TXN000030
25	15	Manish Gupta	25	ACC100025	Fixed Deposit	Deposit	120000	2025-03-10 10:25	Fixed Deposit Credit	TXN000025
16	16	Kavya Iyer	16	ACC100016	Savings	Withdrawal	5000	2025-02-15 14:10	ATM Withdrawal	TXN000016
17	17	Arjun Rao	17	ACC100017	Salary	Deposit	55000	2025-02-18 10:45	Salary Credit	TXN000017
18	18	Meera Patil	18	ACC100018	Savings	Transfer	18000	2025-02-20 13:15	Fund Transfer	TXN000018
19	19	Sameer Khan	19	ACC100019	Current	Deposit	48000	2025-02-22 11:35	Business Deposit	TXN000019
20	20	Tanya Roy	20	ACC100020	Savings	Withdrawal	7500	2025-02-25 16:20	Cash Withdrawal	TXN000020

Step 32 – fund_transfer_summary

The most demanding join in the project. `accounts` and `customers` are each joined twice under separate aliases – `sender_acc / sender` and `receiver_acc / receiver` – so that one transfer row resolves into two named parties.

STEP 32 – CREATE VIEW FUND_TRANSFER_SUMMARY

CREATE VIEW fund_transfer_summary AS
SELECT
 f.transfer_id,
 f.sender_account_id,
 sender.full_name AS sender_name,
 sender_acc.account_number AS sender_account_number,
 f.receiver_account_id,
 receiver.full_name AS receiver_name,
 receiver_acc.account_number AS receiver_account_number,
 f.amount,
 f.transfer_date,
 f.transfer_mode,
 f.transfer_status,
 f.reference_number
FROM fund_transfers f
JOIN accounts sender_acc ON f.sender_account_id = sender_acc.account_id
JOIN customers sender ON sender_acc.customer_id = sender.customer_id
JOIN accounts receiver_acc ON f.receiver_account_id = receiver_acc.account_id
JOIN customers receiver ON receiver_acc.customer_id = receiver.customer_id;

SELECT * FROM fund_transfer_summary;

OUTPUT – 20 ROWS

transfer_id	sender_account_id	sender_name	sender_account_number	receiver_account_id	receiver_name	receiver_account_number	amount	transfer_date	transfer_mode	transfer_status	reference_number
1	1	Aarav Mehta	ACC100001	2	Priya Sharma	ACC100002	5000	2025-01-07 10:30	UPI	Completed	TRF000001
2	2	Priya Sharma	ACC100002	3	Rohan Patil	ACC100003	12000	2025-01-12 11:15	NEFT	Completed	TRF000002
3	3	Rohan Patil	ACC100003	4	Sneha Joshi	ACC100004	25000	2025-01-18 14:20	RTGS	Completed	TRF000003
4	4	Sneha Joshi	ACC100004	5	Vikas Rao	ACC100005	7000	2025-01-22 09:45	UPI	Completed	TRF000004
5	5	Vikas Rao	ACC100005	6	Ananya Shah	ACC100006	15000	2025-01-28 16:10	IMPS	Completed	TRF000005
6	6	Ananya Shah	ACC100006	7	Karan Desai	ACC100007	18000	2025-02-03 10:20	NEFT	Completed	TRF000006
7	7	Karan Desai	ACC100007	8	Neha Kulkarni	ACC100008	22000	2025-02-08 13:30	RTGS	Completed	TRF000007
8	8	Neha Kulkarni	ACC100008	9	Rahul Verma	ACC100009	6500	2025-02-12 15:40	UPI	Completed	TRF000008
9	9	Rahul Verma	ACC100009	10	Pooja Deshmukh	ACC100010	9500	2025-02-16 11:50	IMPS	Completed	TRF000009
10	10	Pooja Deshmukh	ACC100010	11	Aditya Singh	ACC100011	14000	2025-02-20 12:15	NEFT	Completed	TRF000010
11	11	Aditya Singh	ACC100011	12	Riya Kapoor	ACC100012	30000	2025-02-25 14:45	RTGS	Completed	TRF000011
12	12	Riya Kapoor	ACC100012	13	Siddharth Nair	ACC100013	8500	2025-03-01 10:35	UPI	Completed	TRF000012
13	13	Siddharth Nair	ACC100013	14	Isha Malhotra	ACC100014	16000	2025-03-05 13:20	IMPS	Completed	TRF000013
14	14	Isha Malhotra	ACC100014	15	Manish Gupta	ACC100015	11000	2025-03-10 15:10	NEFT	Completed	TRF000014
15	15	Manish Gupta	ACC100015	16	Kavya Iyer	ACC100016	20000	2025-03-15 11:25	RTGS	Completed	TRF000015
16	16	Kavya Iyer	ACC100016	17	Arjun Rao	ACC100017	7500	2025-03-20 09:55	UPI	Completed	TRF000016
17	17	Arjun Rao	ACC100017	18	Meera Patil	ACC100018	13500	2025-03-25 14:30	IMPS	Completed	TRF000017
18	18	Meera Patil	ACC100018	19	Sameer Khan	ACC100019	24000	2025-04-02 12:10	NEFT	Completed	TRF000018
19	19	Sameer Khan	ACC100019	20	Tanya Roy	ACC100020	17500	2025-04-07 16:20	UPI	Completed	TRF000019
20	20	Tanya Roy	ACC100020	1	Aarav Mehta	ACC100001	9000	2025-04-12 10:40	IMPS	Completed	TRF000020

Step 33 – atm_transaction_summary

A four-table join linking the machine, its location, the account and the customer. Note the alias `at` for `atm_transactions` and `a` for `atms` – keeping them distinct is what makes the query readable.

STEP 33 – CREATE VIEW ATM_TRANSACTION_SUMMARY

CREATE VIEW atm_transaction_summary AS
SELECT
 at.atm_transaction_id,
 at.atm_id,
 a.atm_location,
 a.city,
 at.account_id,
 c.full_name,
 acc.account_number,
 acc.account_type,
 at.transaction_type,
 at.amount,
 at.transaction_date,
 at.status,

```
        at.reference_number
FROM atm_transactions at
JOIN atms      a  ON at.atm_id    = a.atm_id
JOIN accounts  acc ON at.account_id = acc.account_id
JOIN customers c  ON acc.customer_id = c.customer_id;

SELECT * FROM atm_transaction_summary;
```

OUTPUT — 25 ROWS

atm_transaction_id	atm_id	atm_location	city	account_id	full_name	account_number	account_type	transaction_type	amount	transaction_date	status	reference_number
1	1	Mumbai Central Railway Road	Mumbai	1	Aarav Mehta	ACC100001	Savings	Withdrawal	5000	2025-01-05 10:15	Successful	ATM000001
11	1	Mumbai Central Railway Road	Mumbai	11	Aditya Singh	ACC100011	Current	Withdrawal	12000	2025-02-12 10:10	Successful	ATM000011
21	1	Mumbai Central Railway Road	Mumbai	21	Aarav Mehta	ACC100021	Fixed Deposit	Withdrawal	15000	2025-03-18 14:25	Successful	ATM000021
2	2	MG Road Shopping Complex	Mumbai	2	Priya Sharma	ACC100002	Savings	Withdrawal	8000	2025-01-08 12:30	Successful	ATM000002
12	2	MG Road Shopping Complex	Mumbai	12	Riya Kapoor	ACC100012	Savings	Deposit	18000	2025-02-15 12:25	Successful	ATM000012
3	3	Andheri West Metro Station	Mumbai	3	Rohan Patil	ACC100003	Current	Deposit	15000	2025-01-12 11:20	Successful	ATM000003
13	3	Andheri West Metro Station	Mumbai	13	Siddharth Nair	ACC100013	Salary	Withdrawal	9000	2025-02-18 14:30	Successful	ATM000013
22	3	Andheri West Metro Station	Mumbai	22	Rohan Patil	ACC100022	Savings	Deposit	17000	2025-03-20 11:35	Successful	ATM000022
4	4	Andheri Market Area	Mumbai	4	Sneha Joshi	ACC100004	Savings	Withdrawal	4000	2025-01-15 15:10	Successful	ATM000004
14	4	Andheri Market Area	Mumbai	14	Isha Malhotra	ACC100014	Savings	Balance Inquiry	0	2025-02-20 15:45	Successful	ATM000014
5	5	Kothrud Main Road	Pune	5	Vikas Rao	ACC100005	Salary	Balance Inquiry	0	2025-01-18 09:45	Successful	ATM000005
15	5	Kothrud Main Road	Pune	15	Manish Gupta	ACC100015	Current	Withdrawal	6500	2025-02-25 11:50	Successful	ATM000015
23	5	Kothrud Main Road	Pune	23	Ananya Shah	ACC100023	Current	Withdrawal	7000	2025-03-22 15:00	Successful	ATM000023
6	6	Pune Camp Area	Pune	6	Ananya Shah	ACC100006	Savings	Withdrawal	10000	2025-01-20 14:20	Successful	ATM000006
16	6	Pune Camp Area	Pune	16	Kavya Iyer	ACC100016	Savings	Deposit	25000	2025-03-01 09:30	Successful	ATM000016
7	7	College Road	Nashik	7	Karan Desai	ACC100007	Current	Deposit	20000	2025-01-25 10:30	Successful	ATM000007
17	7	College Road	Nashik	17	Arjun Rao	ACC100017	Salary	Withdrawal	11000	2025-03-05 13:15	Successful	ATM000017
24	7	College Road	Nashik	24	Aditya Singh	ACC100024	Savings	Withdrawal	8500	2025-03-25 10:50	Successful	ATM000024
8	8	Civil Lines	Nagpur	8	Neha Kulkarni	ACC100008	Savings	Withdrawal	6000	2025-02-02 16:15	Successful	ATM000008
18	8	Civil Lines	Nagpur	18	Meera Patil	ACC100018	Savings	Withdrawal	5500	2025-03-08 16:20	Successful	ATM000018
9	9	Thane West Station Road	Thane	9	Rahul Verma	ACC100009	Salary	Withdrawal	7500	2025-02-05 11:00	Successful	ATM000009
19	9	Thane West Station Road	Thane	19	Sameer Khan	ACC100019	Current	Deposit	22000	2025-03-12 10:40	Successful	ATM000019
25	9	Thane West Station Road	Thane	25	Manish Gupta	ACC100025	Fixed Deposit	Deposit	30000	2025-03-28 13:20	Successful	ATM000025
10	10	Bandra Linking Road	Mumbai	10	Pooja Deshmukh	ACC100010	Savings	Balance Inquiry	0	2025-02-08 13:40	Successful	ATM000010
20	10	Bandra Linking Road	Mumbai	20	Tanya Roy	ACC100020	Savings	Balance Inquiry	0	2025-03-15 12:10	Successful	ATM000020

Step 34 — card_summary

Resolves a card to both its holder and the account it draws on, so a support agent can verify a card in one lookup.

```
STEP 34 — CREATE VIEW CARD_SUMMARY

CREATE VIEW card_summary AS
SELECT
    cd.card_id,
    c.customer_id,
    c.full_name,
    acc.account_id,
    acc.account_number,
    acc.account_type,
    cd.card_type,
    cd.card_number_masked,
    cd.issue_date,
    cd.expiry_date,
    cd.card_status
FROM cards cd
JOIN customers c  ON cd.customer_id = c.customer_id
JOIN accounts  acc ON cd.account_id  = acc.account_id;

SELECT * FROM card_summary;
```

OUTPUT — 20 ROWS

card_id	customer_id	full_name	account_id	account_number	account_type	card_type	card_number_masked	issue_date	expiry_date	card_status
1	1	Aarav Mehta	1	ACC100001	Savings	Debit	XXXX-XXXX-XXXX-1001	2024-01-20	2029-01-20	Active
2	2	Priya Sharma	2	ACC100002	Savings	Debit	XXXX-XXXX-XXXX-1002	2024-01-25	2029-01-25	Active
3	3	Rohan Patil	3	ACC100003	Current	Credit	XXXX-XXXX-XXXX-1003	2024-02-15	2029-02-15	Active
4	4	Sneha Joshi	4	ACC100004	Savings	Debit	XXXX-XXXX-XXXX-1004	2024-02-28	2029-02-28	Active
5	5	Vikas Rao	5	ACC100005	Salary	Debit	XXXX-XXXX-XXXX-1005	2024-03-10	2029-03-10	Active
6	6	Ananya Shah	6	ACC100006	Savings	Credit	XXXX-XXXX-XXXX-1006	2024-03-25	2029-03-25	Active
7	7	Karan Desai	7	ACC100007	Current	Debit	XXXX-XXXX-XXXX-1007	2024-04-10	2029-04-10	Active
8	8	Neha Kulkarni	8	ACC100008	Savings	Debit	XXXX-XXXX-XXXX-1008	2024-04-25	2029-04-25	Active
9	9	Rahul Verma	9	ACC100009	Salary	Credit	XXXX-XXXX-XXXX-1009	2024-05-10	2029-05-10	Active
10	10	Pooja Deshmukh	10	ACC100010	Savings	Debit	XXXX-XXXX-XXXX-1010	2024-05-25	2029-05-25	Active
11	11	Aditya Singh	11	ACC100011	Current	Debit	XXXX-XXXX-XXXX-1011	2024-06-15	2029-06-15	Active
12	12	Riya Kapoor	12	ACC100012	Savings	Credit	XXXX-XXXX-XXXX-1012	2024-06-28	2029-06-28	Active
13	13	Siddharth Nair	13	ACC100013	Salary	Debit	XXXX-XXXX-XXXX-1013	2024-07-12	2029-07-12	Active
14	14	Isha Malhotra	14	ACC100014	Savings	Debit	XXXX-XXXX-XXXX-1014	2024-07-28	2029-07-28	Active
15	15	Manish Gupta	15	ACC100015	Current	Credit	XXXX-XXXX-XXXX-1015	2024-08-12	2029-08-12	Active

card_id	customer_id	full_name	account_id	account_number	account_type	card_type	card_number_masked	issue_date	expiry_date	card_status
16	16	Kavya Iyer	16	ACC100016	Savings	Debit	XXXX-XXXX-XXXX-1016	2024-08-28	2029-08-28	Active
17	17	Arjun Rao	17	ACC100017	Salary	Debit	XXXX-XXXX-XXXX-1017	2024-09-12	2029-09-12	Active
18	18	Meera Patil	18	ACC100018	Savings	Credit	XXXX-XXXX-XXXX-1018	2024-09-28	2029-09-28	Active
19	19	Sameer Khan	19	ACC100019	Current	Debit	XXXX-XXXX-XXXX-1019	2024-10-12	2029-10-12	Active
20	20	Tanya Roy	20	ACC100020	Savings	Debit	XXXX-XXXX-XXXX-1020	2024-10-28	2029-10-28	Active

Step 35 – branch_performance

The first aggregating view. `LEFT JOIN` keeps branches with no accounts in the result, and `COALESCE(SUM(...), 0)` reports their balance as zero rather than `NULL`. `COUNT(DISTINCT a.customer_id)` is essential here: without `DISTINCT`, a customer with two accounts at the same branch would be counted twice.

STEP 35 – CREATE VIEW BRANCH_PERFORMANCE

CREATE VIEW branch_performance AS
SELECT
 b.branch_id,
 b.branch_name,
 b.branch_code,
 b.city,
 b.state,
 b.manager_name,
 COUNT(DISTINCT a.customer_id) AS total_customers,
 COUNT(a.account_id) AS total_accounts,
 COALESCE(SUM(a.balance), 0) AS total_balance
FROM branches b
LEFT JOIN accounts a
 ON b.branch_id = a.branch_id
GROUP BY
 b.branch_id, b.branch_name, b.branch_code,
 b.city, b.state, b.manager_name;

SELECT * FROM branch_performance;

OUTPUT – 8 ROWS

branch_id	branch_name	branch_code	city	state	manager_name	total_customers	total_accounts	total_balance
1	Mumbai Central Branch	MUM001	Mumbai	Maharashtra	Rahul Sharma	3	4	454000
2	Andheri Branch	MUM002	Mumbai	Maharashtra	Priya Mehta	3	3	313000
3	Pune Main Branch	PUN001	Pune	Maharashtra	Amit Patil	4	5	388000
4	Nashik Branch	NSK001	Nashik	Maharashtra	Neha Joshi	2	3	480000
5	Nagpur Branch	NAG001	Nagpur	Maharashtra	Vikas Rao	2	2	139000
6	Thane Branch	THN001	Thane	Maharashtra	Sneha Kulkarni	2	2	258000
7	Bandra Branch	MUM003	Mumbai	Maharashtra	Karan Shah	3	5	604000
8	Aurangabad Branch	AUR001	Aurangabad	Maharashtra	Pooja Deshmukh	1	1	43000

Step 36 – department_employee_summary

Headcount and payroll per department. Again a `LEFT JOIN`, so a newly created department with no staff would still appear with a zero total rather than vanishing.

STEP 36 – CREATE VIEW DEPARTMENT_EMPLOYEE_SUMMARY

DESCRIBE departments;

CREATE VIEW department_employee_summary AS
SELECT
 d.department_id,
 d.department_name,
 COUNT(e.employee_id) AS total_employees,
 COALESCE(SUM(e.salary), 0) AS total_salary,
 COALESCE(AVG(e.salary), 0) AS average_salary
FROM departments d
LEFT JOIN employees e
 ON d.department_id = e.department_id
GROUP BY
 d.department_id,
 d.department_name;

SELECT * FROM department_employee_summary;

OUTPUT – 8 ROWS

department_id	department_name	total_employees	total_salary	average_salary
1	Accounts	3	263000	87666.66667
8	Audit	2	89000	44500
3	Customer Service	3	177000	59000
6	Human Resources	2	133000	66500
5	IT	2	118000	59000
2	Loans	3	139000	46333.33333
4	Operations	3	128000	42666.66667
7	Risk Management	2	95000	47500

OBSERVATION FOR FUTURE WORK

The `employees` table carries both a free-text `department` column and the normalised `department_id`, and the two do not always agree — the three branch managers are labelled "Management" in text but mapped to `department_id = 1` (Accounts). Dropping the redundant text column and driving all reporting from the foreign key would remove the ambiguity. This is discussed further in Section 09.

Step 37 — loan_payment_summary

Combines a join and an aggregate: each loan is matched to its repayments, summed, and the outstanding balance is derived as a calculated column. The `LEFT JOIN` means a loan with no repayments yet still reports a remaining amount equal to the full principal.

STEP 37 — CREATE VIEW LOAN_PAYMENT_SUMMARY

```
DESCRIBE loan_payments;

CREATE VIEW loan_payment_summary AS
SELECT
  l.loan_id,
  c.customer_id,
  c.full_name,
  l.loan_type,
  l.loan_amount,
  COUNT(lp.payment_id)                AS total_payments,
  COALESCE(SUM(lp.payment_amount), 0) AS total_paid,
  (l.loan_amount - COALESCE(SUM(lp.payment_amount), 0)) AS remaining_amount,
  l.loan_status
FROM loans l
JOIN customers c
  ON l.customer_id = c.customer_id
LEFT JOIN loan_payments lp
  ON l.loan_id = lp.loan_id
GROUP BY
  l.loan_id, c.customer_id, c.full_name,
  l.loan_type, l.loan_amount, l.loan_status;

SELECT * FROM loan_payment_summary;
```

OUTPUT — 20 ROWS

loan_id	customer_id	full_name	loan_type	loan_amount	total_payments	total_paid	remaining_amount	loan_status
1	1	Aarav Mehta	Home Loan	250000	1	25000	247500	Active
2	2	Priya Sharma	Personal Loan	500000	1	11000	489000	Active
3	3	Rohan Patil	Business Loan	1500000	1	22000	1478000	Active
4	4	Sneha Joshi	Car Loan	800000	1	14500	785500	Active
5	5	Vikas Rao	Education Loan	600000	1	8500	591500	Active
6	6	Ananya Shah	Home Loan	320000	1	28000	317200	Active
7	7	Karan Desai	Personal Loan	750000	1	16500	733500	Active
8	8	Neha Kulkarni	Car Loan	950000	1	15500	934500	Active
9	9	Rahul Verma	Business Loan	2000000	1	26500	1973500	Active
10	10	Pooja Deshmukh	Education Loan	450000	1	7500	442500	Active
11	11	Aditya Singh	Home Loan	2800000	1	26000	2774000	Active
12	12	Riya Kapoor	Personal Loan	400000	1	12000	388000	Active
13	13	Siddharth Nair	Business Loan	1750000	1	24000	1726000	Active
14	14	Isha Malhotra	Car Loan	700000	1	13500	686500	Active
15	15	Manish Gupta	Home Loan	2200000	1	23500	2176500	Active
16	16	Kavya Iyer	Personal Loan	550000	1	12500	537500	Active
17	17	Arjun Rao	Business Loan	1250000	1	21000	1229000	Active
18	18	Meera Patil	Education Loan	500000	1	9000	491000	Active
19	19	Sameer Khan	Car Loan	900000	1	15000	885000	Active
20	20	Tanya Roy	Home Loan	3000000	1	27000	2973000	Active

Step 38 — customer_financial_summary

The keystone view: one row per customer combining deposits and borrowings. Because a customer can have several accounts *and* several loans, the two `LEFT JOIN` s multiply rows against each other — so `COUNT(DISTINCT ...)` and `SUM(DISTINCT ...)` are used to prevent the fan-out from inflating the totals. Six of the seven queries in Section 07 read from this view.

STEP 38 — CREATE VIEW CUSTOMER_FINANCIAL_SUMMARY

```
CREATE VIEW customer_financial_summary AS
SELECT
  c.customer_id,
  c.full_name,
  COUNT(DISTINCT a.account_id)                AS total_accounts,
  COALESCE(SUM(DISTINCT a.balance), 0)         AS total_account_balance,
  COUNT(DISTINCT l.loan_id)                  AS total_loans,
  COALESCE(SUM(DISTINCT l.loan_amount), 0)     AS total_loan_amount
FROM customers c
LEFT JOIN accounts a
  ON c.customer_id = a.customer_id
LEFT JOIN loans l
  ON c.customer_id = l.customer_id
GROUP BY
  c.customer_id,
  c.full_name;
```

```
SELECT * FROM customer_financial_summary;
```

OUTPUT — 20 ROWS

customer_id	full_name	total_accounts	total_account_balance	total_loans	total_loan_amount
1	Aarav Mehta	2	335000	1	2500000
2	Priya Sharma	1	62000	1	500000
3	Rohan Patil	2	190000	1	1500000
4	Sneha Joshi	1	48000	1	800000
5	Vikas Rao	1	72000	1	600000
6	Ananya Shah	2	211000	1	3200000
7	Karan Desai	1	185000	1	750000
8	Neha Kulkarni	1	56000	1	950000
9	Rahul Verma	1	68000	1	2000000
10	Pooja Deshmukh	1	43000	1	450000
11	Aditya Singh	2	298000	1	2800000
12	Riya Kapoor	1	76000	1	400000
13	Siddharth Nair	1	83000	1	1750000
14	Isha Malhotra	1	51000	1	700000
15	Manish Gupta	2	432000	1	2200000
16	Kavya Iyer	1	67000	1	550000
17	Arjun Rao	1	95000	1	1250000
18	Meera Patil	1	59000	1	500000
19	Sameer Khan	1	175000	1	900000
20	Tanya Roy	1	73000	1	3000000

WHY SUM(DISTINCT ...) MATTERS HERE

With two `LEFT JOIN` s in one query, a customer holding two accounts and one loan produces two intermediate rows, and the loan amount would otherwise be counted twice. `DISTINCT` suppresses that. The trade-off worth knowing: if the same customer ever held two accounts with an *identical* balance, `SUM(DISTINCT balance)` would count that value once. A correlated subquery or a pre-aggregated derived table would be the production-grade fix.

Dashboard and Reporting Queries

Steps 39–49: eleven analytical queries that turn the views into the numbers a branch manager or credit officer would actually look at.

Each query below is short precisely because the joins were pushed down into the views. This is the payoff of the previous section: business questions become three-line statements.

Step 39 — Top 5 customers by account balance

STEP 39 — HIGHEST DEPOSIT RELATIONSHIPS

```
SELECT
  customer_id,
  full_name,
  total_accounts,
  total_account_balance
FROM customer_financial_summary
ORDER BY total_account_balance DESC
LIMIT 5;
```

OUTPUT — 5 ROWS

customer_id	full_name	total_accounts	total_account_balance
15	Manish Gupta	2	432000
1	Aarav Mehta	2	335000
11	Aditya Singh	2	298000
6	Ananya Shah	2	211000
3	Rohan Patil	2	190000

Every customer in the top five holds two accounts. The pattern is clear in the data: the second account — usually a Fixed Deposit or a Current account — is what moves a customer into the high-value band, not a larger single balance.

Step 40 — Top 5 customers by loan amount

STEP 40 — LARGEST CREDIT EXPOSURES

```
SELECT
  customer_id,
  full_name,
  total_loans,
  total_loan_amount
FROM customer_financial_summary
ORDER BY total_loan_amount DESC
LIMIT 5;
```

OUTPUT — 5 ROWS

customer_id	full_name	total_loans	total_loan_amount
6	Ananya Shah	1	3200000
20	Tanya Roy	1	3000000
11	Aditya Singh	1	2800000
1	Aarav Mehta	1	2500000
15	Manish Gupta	1	2200000

All five are home loans, which is what a mortgage-weighted retail book looks like. Note that Tanya Roy carries the second-largest exposure in the bank against a deposit balance of only ₹73,000 — exactly the kind of imbalance a risk report exists to surface.

Step 41 — Loan type analysis

STEP 41 — PRODUCT MIX OF THE LOAN BOOK

```
SELECT
  loan_type,
  COUNT(*)          AS total_loans,
  SUM(loan_amount)  AS total_loan_amount,
  AVG(loan_amount)  AS average_loan_amount
FROM loans
GROUP BY loan_type
ORDER BY total_loan_amount DESC;
```

OUTPUT — 5 ROWS

loan_type	total_loans	total_loan_amount	average_loan_amount
Home Loan	5	13700000	2740000
Business Loan	4	6500000	1625000
Car Loan	4	3350000	837500
Personal Loan	4	2200000	550000
Education Loan	3	1550000	516666.6667

Home loans are a quarter of the accounts but half the money: ₹13.7 million of a ₹27.3 million book, at an average ticket of ₹2.74 million — more than five times the average education loan.

Step 42 — Transaction type analysis

STEP 42 — LEDGER ACTIVITY BY TYPE

SELECT
transaction_type,
COUNT(*) AS total_transactions,
SUM(amount) AS total_amount,
AVG(amount) AS average_amount
FROM transactions
GROUP BY transaction_type
ORDER BY total_amount DESC;

OUTPUT — 3 ROWS

transaction_type	total_transactions	total_amount	average_amount
Deposit	15	731000	48733.33333
Transfer	5	92000	18400
Withdrawal	10	73500	7350

Deposits average ₹48,733 against withdrawals of ₹7,350 — money enters in large lumps (salary credits, fixed deposits, business takings) and leaves in small ones.

Step 43 — Fund transfer mode analysis

STEP 43 — PAYMENT RAIL USAGE

SELECT
transfer_mode,
COUNT(*) AS total_transfers,
SUM(amount) AS total_amount,
AVG(amount) AS average_amount
FROM fund_transfers
GROUP BY transfer_mode
ORDER BY total_amount DESC;

OUTPUT — 4 ROWS

transfer_mode	total_transfers	total_amount	average_amount
RTGS	4	97000	24250
NEFT	5	79000	15800
IMPS	5	63000	12600
UPI	6	52000	8666.66667

The rails sort themselves by ticket size exactly as they do in practice: UPI is the most frequent and the smallest at ₹8,667 average, while RTGS carries the fewest transfers but the largest value at ₹24,250.

Step 44 — ATM transaction analysis

STEP 44 — ATM ACTIVITY BY TYPE

SELECT
transaction_type,
COUNT(*) AS total_transactions,
SUM(amount) AS total_amount,
AVG(amount) AS average_amount
FROM atm_transactions
GROUP BY transaction_type
ORDER BY total_amount DESC;

OUTPUT — 3 ROWS

transaction_type	total_transactions	total_amount	average_amount
Deposit	7	147000	21000
Withdrawal	14	115000	8214.285714
Balance Inquiry	4	0	0

Withdrawals are twice as frequent as deposits but move less money. Balance enquiries are counted with a zero value rather than excluded — useful for machine-utilisation reporting, since an enquiry still occupies the ATM.

Step 45 — Branch-wise account balance analysis

STEP 45 — DEPOSITS HELD PER BRANCH

SELECT
branch_name,
city,
total_accounts,
total_customers,
total_balance
FROM branch_performance
ORDER BY total_balance DESC;

OUTPUT — 8 ROWS

branch_name	city	total_accounts	total_customers	total_balance
Bandra Branch	Mumbai	5	3	604000
Nashik Branch	Nashik	3	2	480000
Mumbai Central Branch	Mumbai	4	3	454000
Pune Main Branch	Pune	5	4	388000
Andheri Branch	Mumbai	3	3	313000
Thane Branch	Thane	2	2	258000
Nagpur Branch	Nagpur	2	2	139000
Aurangabad Branch	Aurangabad	1	1	43000

Account count does not determine deposits. Pune Main holds five accounts across four customers but ₹388,000, while Nashik holds three accounts across two customers and ₹480,000 — the difference is a single ₹300,000 fixed deposit. Deposit value per customer, not customer count, is the meaningful branch metric.

Step 46 — Department salary analysis

STEP 46 — PAYROLL DISTRIBUTION

SELECT
 department_name,
 total_employees,
 total_salary,
 average_salary
FROM department_employee_summary
ORDER BY total_salary DESC;

OUTPUT — 8 ROWS

department_name	total_employees	total_salary	average_salary
Accounts	3	263000	87666.66667
Customer Service	3	177000	59000
Loans	3	139000	46333.33333
Human Resources	2	133000	66500
Operations	3	128000	42666.66667
IT	2	118000	59000
Risk Management	2	95000	47500
Audit	2	89000	44500

Total monthly payroll across the network is ₹1,142,000 for twenty staff. The mapping quirk noted in Section 06 shows up here too: the three branch managers sit under `department_id = 1`, which is why "Accounts" reports the highest average salary.

Step 47 — Loan payment analysis

This query aggregates a view that was itself an aggregate — the per-loan totals from `loan_payment_summary` are rolled up a second time by product.

STEP 47 — RECOVERY POSITION BY PRODUCT

SELECT
 loan_type,
 COUNT(*) AS total_loans,
 SUM(loan_amount) AS total_loan_amount,
 SUM(total_paid) AS total_paid,
 SUM(remaining_amount) AS total_remaining
FROM loan_payment_summary
GROUP BY loan_type
ORDER BY total_loan_amount DESC;

OUTPUT — 5 ROWS

loan_type	total_loans	total_loan_amount	total_paid	total_remaining
Home Loan	5	13700000	129500	13570500
Business Loan	4	6500000	93500	6406500
Car Loan	4	3350000	58500	3291500
Personal Loan	4	2200000	52000	2148000
Education Loan	3	1550000	25000	1525000

Across the whole book, ₹358,500 has been repaid against ₹27.3 million disbursed. With only one instalment recorded per loan that is expected — the value of the query is that it gives the recovery position per product in a single statement.

Step 48 — Customer balance ranking (window function)

`RANK() OVER (ORDER BY ...)` assigns a position to every row without collapsing the result set, which a `GROUP BY` cannot do. Ties would share a rank and skip the next value — visible in Step 49 below.

STEP 48 — RANK() OVER DEPOSIT BALANCE

```
SELECT
    customer_id,
    full_name,
    total_accounts,
    total_account_balance,
    RANK() OVER (ORDER BY total_account_balance DESC) AS balance_rank
FROM customer_financial_summary
ORDER BY balance_rank;
```

OUTPUT — 20 ROWS

customer_id		full_name	total_accounts	total_account_balance	balance_rank
	15	Manish Gupta	2	432000	1
	1	Aarav Mehta	2	335000	2
	11	Aditya Singh	2	298000	3
	6	Ananya Shah	2	211000	4
	3	Rohan Patil	2	190000	5
	7	Karan Desai	1	185000	6
	19	Sameer Khan	1	175000	7
	17	Arjun Rao	1	95000	8
	13	Siddharth Nair	1	83000	9
	12	Riya Kapoor	1	76000	10
	20	Tanya Roy	1	73000	11
	5	Vikas Rao	1	72000	12
	9	Rahul Verma	1	68000	13
	16	Kavya Iyer	1	67000	14
	2	Priya Sharma	1	62000	15
	18	Meera Patil	1	59000	16
	8	Neha Kulkarni	1	56000	17
	14	Isha Malhotra	1	51000	18
	4	Sneha Joshi	1	48000	19
	10	Pooja Deshmukh	1	43000	20

Step 49 — Loan amount ranking (window function)

STEP 49 — RANK() OVER CREDIT EXPOSURE

```
SELECT
    customer_id,
    full_name,
    total_loans,
    total_loan_amount,
    RANK() OVER (
        ORDER BY total_loan_amount DESC
    ) AS loan_rank
FROM customer_financial_summary
ORDER BY loan_rank;
```

OUTPUT — 20 ROWS

customer_id		full_name	total_loans	total_loan_amount	loan_rank
	6	Ananya Shah	1	3200000	1
	20	Tanya Roy	1	3000000	2
	11	Aditya Singh	1	2800000	3
	1	Aarav Mehta	1	2500000	4
	15	Manish Gupta	1	2200000	5
	9	Rahul Verma	1	2000000	6
	13	Siddharth Nair	1	1750000	7
	3	Rohan Patil	1	1500000	8
	17	Arjun Rao	1	1250000	9
	8	Neha Kulkarni	1	950000	10
	19	Sameer Khan	1	900000	11
	4	Sneha Joshi	1	800000	12
	7	Karan Desai	1	750000	13
	14	Isha Malhotra	1	700000	14
	5	Vikas Rao	1	600000	15
	16	Kavya Iyer	1	550000	16
	2	Priya Sharma	1	500000	17
	18	Meera Patil	1	500000	17
	10	Pooja Deshmukh	1	450000	19
	12	Riya Kapoor	1	400000	20

TIE HANDLING

Priya Sharma and Meera Patil both hold ₹500,000 and therefore share rank 17. Rank 18 is then skipped and the next customer is ranked 19 — standard `RANK()` behaviour. `DENSE_RANK()` would have continued at 18, and `ROW_NUMBER()` would have broken the tie arbitrarily.

Step 51 — Duplicate loan check

A data-quality guard rather than a report. It groups loans on every business attribute and returns only groups appearing more than once, which would catch a loan application submitted twice.

STEP 51 — HAVING COUNT(*) > 1

```
SELECT
  customer_id,
  branch_id,
  loan_type,
  loan_amount,
  interest_rate,
  tenure_months,
  start_date,
  end_date,
  loan_status,
  COUNT(*) AS duplicate_count
FROM loans
GROUP BY
  customer_id, branch_id, loan_type, loan_amount, interest_rate,
  tenure_months, start_date, end_date, loan_status
HAVING COUNT(*) > 1;
```

RESULT: EMPTY SET

The query returns no rows, which is the correct and desired outcome — the loan book contains no duplicates. `HAVING` is used rather than `WHERE` because the filter is applied to the aggregate `COUNT(*)` after grouping, not to individual rows before it.

Stored Procedures

Steps 50–63: fourteen parameterised routines that answer every common customer-level question from a single input — the customer ID.

Each procedure follows the same discipline: drop if it already exists so the script can be re-run safely, switch the statement delimiter to `$$` so that semicolons inside the body are not treated as the end of the definition, take one `IN` parameter, and restore the delimiter afterwards. Every procedure is shown below with a live call against customer 1, Aarav Mehta.

Procedure	Step	Returns
GetCustomerLoans	50	Every loan held by the customer
GetCustomerLoanCount	51a	Number of loans
GetCustomerLoanAmount	52	Total principal borrowed
GetCustomerAccountCount	53	Number of accounts held
GetCustomerBalance	54	Combined deposit balance
GetCustomerFinancialSummary	55	Accounts, balance, loans and exposure in one row
GetCustomerTransactions	56	Full ledger history, date ordered
GetCustomerTransfers	57	Transfers sent or received
GetCustomerLoanPayments	58	Repayment history across all loans
GetCustomerCards	59	Cards with the accounts they draw on
GetCustomerATMTransactions	60	ATM activity with machine location
GetCustomerBeneficiaries	61	Saved external payees
GetCustomerBranchDetails	62	Distinct branches the customer banks at
GetCustomerProfile	63	Complete profile with every account

Step 50 — GetCustomerLoans

STEP 50 — GETCUSTOMERLOANS

```
DROP PROCEDURE IF EXISTS GetCustomerLoans;

DELIMITER $$

CREATE PROCEDURE GetCustomerLoans(IN p_customer_id INT)
BEGIN
  SELECT
    loan_id,
    customer_id,
    loan_type,
    loan_amount,
    interest_rate,
    tenure_months,
    start_date,
    end_date,
    loan_status
  FROM loans
  WHERE customer_id = p_customer_id;
END $$

DELIMITER ;

USE banking_system;
CALL GetCustomerLoans(1);
```

OUTPUT — CALL GETCUSTOMERLOANS(1)

loan_id	customer_id	branch_id	loan_type	loan_amount	interest_rate	tenure_months	start_date	end_date	loan_status
1	1	1	Home Loan	2500000	8.50	240	2024-01-15	2044-01-15	Active

The script also verifies the routine is registered with the server:

VERIFICATION — PROCEDURE REGISTRY

```
SHOW PROCEDURE STATUS
WHERE Db = 'banking_system';

SHOW CREATE PROCEDURE GetCustomerLoans;
```

Step 51a — GetCustomerLoanCount

STEP 51A — GETCUSTOMERLOANCOUNT

```
DROP PROCEDURE IF EXISTS GetCustomerLoanCount;

DELIMITER $$

CREATE PROCEDURE GetCustomerLoanCount(IN p_customer_id INT)
```

```

BEGIN
    SELECT
        customer_id,
        COUNT(*) AS total_loans
    FROM loans
    WHERE customer_id = p_customer_id
    GROUP BY customer_id;
END $$

DELIMITER ;

USE banking_system;
CALL GetCustomerLoanCount(1);

```

OUTPUT — CALL GETCUSTOMERLOANCOUNT(1)

customer_id	total_loans
1	1

Step 52 — GetCustomerLoanAmount

STEP 52 — GETCUSTOMERLOANAMOUNT

```

DROP PROCEDURE IF EXISTS GetCustomerLoanAmount;

DELIMITER $$

CREATE PROCEDURE GetCustomerLoanAmount(IN p_customer_id INT)
BEGIN
    SELECT
        customer_id,
        SUM(loan_amount) AS total_loan_amount
    FROM loans
    WHERE customer_id = p_customer_id
    GROUP BY customer_id;
END $$

DELIMITER ;

CALL GetCustomerLoanAmount(1);

```

OUTPUT — CALL GETCUSTOMERLOANAMOUNT(1)

customer_id	total_loan_amount
1	2500000

Step 53 — GetCustomerAccountCount

STEP 53 — GETCUSTOMERACCOUNTCOUNT

```

DROP PROCEDURE IF EXISTS GetCustomerAccountCount;

DELIMITER $$

CREATE PROCEDURE GetCustomerAccountCount(IN p_customer_id INT)
BEGIN
    SELECT
        customer_id,
        COUNT(*) AS total_accounts
    FROM accounts
    WHERE customer_id = p_customer_id
    GROUP BY customer_id;
END $$

DELIMITER ;

USE banking_system;
CALL GetCustomerAccountCount(1);

```

OUTPUT — CALL GETCUSTOMERACCOUNTCOUNT(1)

customer_id	total_accounts
1	2

Step 54 — GetCustomerBalance

STEP 54 — GETCUSTOMERBALANCE

```

DROP PROCEDURE IF EXISTS GetCustomerBalance;

DELIMITER $$

CREATE PROCEDURE GetCustomerBalance(IN p_customer_id INT)
BEGIN
    SELECT
        customer_id,
        SUM(balance) AS total_balance
    FROM accounts
    WHERE customer_id = p_customer_id
    GROUP BY customer_id;
END $$

```

```

DELIMITER ;

USE banking_system;
CALL GetCustomerBalance(1);

```

OUTPUT — CALL GETCUSTOMERBALANCE(1)

customer_id	total_balance
1	335000

₹85,000 in the savings account plus ₹250,000 in the fixed deposit — matching the figure reported independently by `customer_financial_summary`.

Step 55 — GetCustomerFinancialSummary

The procedural equivalent of the keystone view, filtered to a single customer. This is the routine quoted in the project summary as `CALL GetCustomerFinancialSummary(1);`

```

STEP 55 — GETCUSTOMERFINANCIALSUMMARY

DROP PROCEDURE IF EXISTS GetCustomerFinancialSummary;

DELIMITER $$

CREATE PROCEDURE GetCustomerFinancialSummary(IN p_customer_id INT)
BEGIN
    SELECT
        c.customer_id,
        c.full_name,
        COUNT(DISTINCT a.account_id)      AS total_accounts,
        COALESCE(SUM(DISTINCT a.balance), 0) AS total_balance,
        COUNT(DISTINCT l.loan_id)         AS total_loans,
        COALESCE(SUM(DISTINCT l.loan_amount), 0) AS total_loan_amount
    FROM customers c
    LEFT JOIN accounts a
        ON c.customer_id = a.customer_id
    LEFT JOIN loans l
        ON c.customer_id = l.customer_id
    WHERE c.customer_id = p_customer_id
    GROUP BY
        c.customer_id,
        c.full_name;
END $$

DELIMITER ;

USE banking_system;
CALL GetCustomerFinancialSummary(1);

```

OUTPUT — CALL GETCUSTOMERFINANCIALSUMMARY(1)

customer_id	full_name	total_accounts	total_balance	total_loans	total_loan_amount
1	Aarav Mehta	2	335000	1	2500000

Step 56 — GetCustomerTransactions

```

STEP 56 — GETCUSTOMERTRANSACTIONS

DROP PROCEDURE IF EXISTS GetCustomerTransactions;

DELIMITER $$

CREATE PROCEDURE GetCustomerTransactions(IN p_customer_id INT)
BEGIN
    SELECT
        t.transaction_id,
        c.customer_id,
        c.full_name,
        a.account_number,
        a.account_type,
        t.transaction_type,
        t.amount,
        t.transaction_date,
        t.description,
        t.reference_number
    FROM transactions t
    JOIN accounts a
        ON t.account_id = a.account_id
    JOIN customers c
        ON a.customer_id = c.customer_id
    WHERE c.customer_id = p_customer_id
    ORDER BY t.transaction_date;
END $$

DELIMITER ;

USE banking_system;
CALL GetCustomerTransactions(1);

```

OUTPUT — CALL GETCUSTOMERTRANSACTIONS(1)

transaction_id	customer_id	full_name	account_number	account_type	transaction_type	amount	transaction_date	description	reference_number
1	1	Aarav Mehta	ACC100001	Savings	Deposit	25000	2025-01-05 10:15	Cash Deposit	TXN000001
21	1	Aarav Mehta	ACC100021	Fixed Deposit	Deposit	100000	2025-03-01 10:05	Fixed Deposit Credit	TXN000021

transaction_id	customer_id	full_name	account_number	account_type	transaction_type	amount	transaction_date	description	reference_number
26	1	Aarav Mehta	ACC100001	Savings	Withdrawal	10000	2025-03-12 15:10	ATM Withdrawal	TXN000026

Step 57 – GetCustomerTransfers

The `OR` in the `WHERE` clause is what makes this useful: the customer appears whether they sent the money or received it, so one call returns a complete payment history rather than half of it.

STEP 57 – GETCUSTOMERTRANSFERS

```
DROP PROCEDURE IF EXISTS GetCustomerTransfers;

DELIMITER $$

CREATE PROCEDURE GetCustomerTransfers(IN p_customer_id INT)
BEGIN
    SELECT
        f.transfer_id,
        f.sender_account_id,
        sender.full_name           AS sender_name,
        sender_acc.account_number AS sender_account_number,
        f.receiver_account_id,
        receiver.full_name         AS receiver_name,
        receiver_acc.account_number AS receiver_account_number,
        f.amount,
        f.transfer_date,
        f.transfer_mode,
        f.transfer_status,
        f.reference_number
    FROM fund_transfers f
    JOIN accounts sender_acc  ON f.sender_account_id = sender_acc.account_id
    JOIN customers sender     ON sender_acc.customer_id = sender.customer_id
    JOIN accounts receiver_acc ON f.receiver_account_id = receiver_acc.account_id
    JOIN customers receiver   ON receiver_acc.customer_id = receiver.customer_id
    WHERE sender.customer_id = p_customer_id
       OR receiver.customer_id = p_customer_id
    ORDER BY f.transfer_date;
END $$

DELIMITER ;

USE banking_system;
CALL GetCustomerTransfers(1);
```

OUTPUT — CALL GETCUSTOMERTRANSFERS(1)

transfer_id	sender_account_id	sender_name	sender_account_number	receiver_account_id	receiver_name	receiver_account_number	amount	transfer_date	transfer_mode	transfer_status	reference_number
1	1	Aarav Mehta	ACC100001	2	Priya Sharma	ACC100002	5000	2025-01-07 10:30	UPI	Completed	TRF000001
20	20	Tanya Roy	ACC100020	1	Aarav Mehta	ACC100001	9000	2025-04-12 10:40	IMPS	Completed	TRF000020

Step 58 – GetCustomerLoanPayments

STEP 58 – GETCUSTOMERLOANPAYMENTS

```
DROP PROCEDURE IF EXISTS GetCustomerLoanPayments;

DELIMITER $$

CREATE PROCEDURE GetCustomerLoanPayments(IN p_customer_id INT)
BEGIN
    SELECT
        lp.payment_id,
        l.loan_id,
        c.customer_id,
        c.full_name,
        l.loan_type,
        l.loan_amount,
        lp.payment_date,
        lp.payment_amount,
        lp.payment_method,
        lp.payment_status,
        lp.reference_number
    FROM loan_payments lp
    JOIN loans l
        ON lp.loan_id = l.loan_id
    JOIN customers c
        ON l.customer_id = c.customer_id
    WHERE c.customer_id = p_customer_id
    ORDER BY lp.payment_date;
END $$

DELIMITER ;

USE banking_system;
CALL GetCustomerLoanPayments(1);
```

OUTPUT — CALL GETCUSTOMERLOANPAYMENTS(1)

payment_id	loan_id	customer_id	full_name	loan_type	loan_amount	payment_date	payment_amount	payment_method	payment_status	reference_number
1	1	1	Aarav Mehta	Home Loan	2500000	2025-01-10	25000	Bank Transfer	Paid	LP000001

Step 59 — GetCustomerCards

STEP 59 — GETCUSTOMERCARDS

```
DROP PROCEDURE IF EXISTS GetCustomerCards;

DELIMITER $$

CREATE PROCEDURE GetCustomerCards(IN p_customer_id INT)
BEGIN
    SELECT
        cd.card_id,
        c.customer_id,
        c.full_name,
        acc.account_number,
        acc.account_type,
        cd.card_type,
        cd.card_number_masked,
        cd.issue_date,
        cd.expiry_date,
        cd.card_status
    FROM cards cd
    JOIN customers c
        ON cd.customer_id = c.customer_id
    JOIN accounts acc
        ON cd.account_id = acc.account_id
    WHERE c.customer_id = p_customer_id
    ORDER BY cd.card_id;
END $$

DELIMITER ;

USE banking_system;
CALL GetCustomerCards(1);
```

OUTPUT — CALL GETCUSTOMERCARDS(1)

card_id	customer_id	full_name	account_number	account_type	card_type	card_number_masked	issue_date	expiry_date	card_status
1	1	Aarav Mehta	ACC100001	Savings	Debit	XXXX-XXXX-XXXX-1001	2024-01-20	2029-01-20	Active

Step 60 — GetCustomerATMTransactions

STEP 60 — GETCUSTOMERATMTRANSACTIONS

```
DROP PROCEDURE IF EXISTS GetCustomerATMTransactions;

DELIMITER $$

CREATE PROCEDURE GetCustomerATMTransactions(IN p_customer_id INT)
BEGIN
    SELECT
        at.atm_transaction_id,
        c.customer_id,
        c.full_name,
        at.atm_id,
        atm.atm_location,
        atm.city,
        acc.account_number,
        at.transaction_type,
        at.amount,
        at.transaction_date,
        at.status,
        at.reference_number
    FROM atm_transactions at
    JOIN accounts acc
        ON at.account_id = acc.account_id
    JOIN customers c
        ON acc.customer_id = c.customer_id
    JOIN atms atm
        ON at.atm_id = atm.atm_id
    WHERE c.customer_id = p_customer_id
    ORDER BY at.transaction_date;
END $$

DELIMITER ;

USE banking_system;
CALL GetCustomerATMTransactions(1);
```

OUTPUT — CALL GETCUSTOMERATMTRANSACTIONS(1)

atm_transaction_id	customer_id	full_name	atm_id	atm_location	city	account_number	transaction_type	amount	transaction_date	status	reference_number
1	1	Aarav Mehta	1	Mumbai Central Railway Road	Mumbai	ACC100001	Withdrawal	5000	2025-01-05 10:15	Successful	ATM000001
21	1	Aarav Mehta	1	Mumbai Central Railway Road	Mumbai	ACC100021	Withdrawal	15000	2025-03-18 14:25	Successful	ATM000021

Step 61 — GetCustomerBeneficiaries

STEP 61 — GETCUSTOMERBENEFICIARIES

```
DROP PROCEDURE IF EXISTS GetCustomerBeneficiaries;

DELIMITER $$

CREATE PROCEDURE GetCustomerBeneficiaries(IN p_customer_id INT)
```

```

BEGIN
    SELECT
        beneficiary_id,
        customer_id,
        beneficiary_name,
        beneficiary_account_number,
        beneficiary_bank_name,
        ifsc_code,
        relationship,
        added_date,
        status
    FROM beneficiaries
    WHERE customer_id = p_customer_id
    ORDER BY beneficiary_id;
END $$

DELIMITER ;

USE banking_system;
CALL GetCustomerBeneficiaries(1);

```

OUTPUT — CALL GETCUSTOMERBENEFICIARIES(1)

beneficiary_id	customer_id	beneficiary_name	beneficiary_account_number	beneficiary_bank_name	ifsc_code	relationship	added_date	status
1	1	Rahul Mehta	BENACC10001	ABC Bank	ABCB0001001	Brother	2025-01-05	Active

Step 62 — GetCustomerBranchDetails

`SELECT DISTINCT` matters here: a customer with two accounts at the same branch would otherwise see that branch listed twice.

STEP 62 — GETCUSTOMERBRANCHDETAILS

```

DROP PROCEDURE IF EXISTS GetCustomerBranchDetails;

DELIMITER $$

CREATE PROCEDURE GetCustomerBranchDetails(IN p_customer_id INT)
BEGIN
    SELECT DISTINCT
        c.customer_id,
        c.full_name,
        b.branch_id,
        b.branch_name,
        b.branch_code,
        b.city,
        b.state,
        b.manager_name,
        b.contact_number,
        b.status
    FROM customers c
    JOIN accounts a
        ON c.customer_id = a.customer_id
    JOIN branches b
        ON a.branch_id = b.branch_id
    WHERE c.customer_id = p_customer_id
    ORDER BY b.branch_id;
END $$

DELIMITER ;

USE banking_system;
CALL GetCustomerBranchDetails(1);

```

OUTPUT — CALL GETCUSTOMERBRANCHDETAILS(1)

customer_id	full_name	branch_id	branch_name	branch_code	city	state	manager_name	contact_number	status
1	Aarav Mehta	1	Mumbai Central Branch	MUM001	Mumbai	Maharashtra	Rahul Sharma	9000000001	Active

Aarav Mehta holds two accounts, both at Mumbai Central, so `DISTINCT` collapses them into the single branch row shown.

Step 63 — GetCustomerProfile

The broadest routine in the set. A `LEFT JOIN` is used deliberately — a newly registered customer with no account yet still returns their profile, with the account columns null, instead of returning nothing at all.

STEP 63 — GETCUSTOMERPROFILE

```

DROP PROCEDURE IF EXISTS GetCustomerProfile;

DELIMITER $$

CREATE PROCEDURE GetCustomerProfile(IN p_customer_id INT)
BEGIN
    SELECT
        c.customer_id,
        c.full_name,
        c.date_of_birth,
        c.gender,
        c.phone_number,
        c.email,
        c.address,
        c.city,
        c.state,
        c.registration_date,
        c.status AS customer_status,

```

```

        a.account_id,
        a.account_number,
        a.account_type,
        a.balance,
        a.interest_rate,
        a.opening_date,
        a.status AS account_status
FROM customers c
LEFT JOIN accounts a
    ON c.customer_id = a.customer_id
WHERE c.customer_id = p_customer_id
ORDER BY a.account_id;
END $$

DELIMITER ;

USE banking_system;
CALL GetCustomerProfile(1);
```

OUTPUT — CALL GETCUSTOMERPROFILE(1)

customer_id	full_name	date_of_birth	gender	phone_number	email	city	registration_date	customer_status	account_id	account_number	account_type	balance	interest_rate	opening_date	account_status
1	Aarav Mehta	1995-04-12	Male	9100000001	aarav@example.com	Mumbai	2024-01-10	Active	1	ACC100001	Savings	85000	4.00	2024-01-12	Active
1	Aarav Mehta	1995-04-12	Male	9100000001	aarav@example.com	Mumbai	2024-01-10	Active	21	ACC100021	Fixed Deposit	250000	7.00	2024-02-15	Active

Code Review and Corrections

One blocking defect and three design observations found by reading the script end to end, each with the correction applied.

Documenting what is wrong with your own code, and fixing it, says more about engineering judgement than a script that claims to be flawless. The issues below were identified by walking the script in execution order and asking what MySQL would do with each statement.

Defect 1 — fund_transfers is created and populated twice

SEVERITY: BLOCKING STATUS: FIXED

The table is created at Step 6 and populated at Step 21. Steps 21A and 21B then repeat both statements verbatim. Executing the script top to bottom therefore fails twice:

Statement	Error	What actually happens
Step 21A CREATE TABLE	ERROR 1050 (42S01)	Table 'fund_transfers' already exists. The statement aborts; the existing table is left untouched.
Step 21B INSERT	ERROR 1062 (23000)	Duplicate entry 'TRF000001' for key 'reference_number'. The <code>UNIQUE</code> constraint rejects the batch, so no duplicate rows are written.

THE CONSTRAINT DID ITS JOB

It is worth noticing why the second insert fails safely. Because `reference_number` was declared `UNIQUE` at Step 6, the database itself refused twenty duplicate payment records. Without that constraint the script would have completed without any error and silently doubled every transfer — `fund_transfer_summary` would report 40 rows, and the mode analysis in Step 43 would report ₹582,000 of transfers instead of ₹291,000. This is the practical argument for declaring constraints at the schema level rather than trusting application code.

THE FIX

Delete Steps 21A and 21B entirely; Steps 6 and 21 already do the work. If the script must remain re-runnable from the top, guard the creation instead:

CORRECTION — DEFECT 1

```
-- Option A (preferred): delete Steps 21A and 21B. Steps 6 and 21 are sufficient.

-- Option B: make the script idempotent
CREATE TABLE IF NOT EXISTS fund_transfers (
  transfer_id          INT PRIMARY KEY AUTO_INCREMENT,
  sender_account_id    INT NOT NULL,
  receiver_account_id  INT NOT NULL,
  amount               DECIMAL(15,2) NOT NULL,
  transfer_date         DATETIME DEFAULT CURRENT_TIMESTAMP,
  transfer_mode         VARCHAR(30),
  transfer_status       VARCHAR(20) DEFAULT 'Completed',
  reference_number      VARCHAR(50) UNIQUE,
  FOREIGN KEY (sender_account_id) REFERENCES accounts(account_id),
  FOREIGN KEY (receiver_account_id) REFERENCES accounts(account_id)
);

-- and make the re-load a no-op rather than an error
INSERT IGNORE INTO fund_transfers
(sender_account_id, receiver_account_id, amount, transfer_date,
 transfer_mode, transfer_status, reference_number)
VALUES
(1, 2, 5000.00, '2025-01-07 10:30:00', 'UPI', 'Completed', 'TRF000001');
-- ... remaining rows unchanged
```

The corrected script in this document applies Option A: the duplicate block is removed, and the schema builds cleanly from an empty server in a single pass.

Defect 2 — employees stores its department twice

SEVERITY: DESIGN STATUS: DOCUMENTED

`employees` carries a free-text `department` column from Step 12 and a normalised `department_id` foreign key from Step 15. The two disagree in the seeded data — staff labelled “Management”, “Finance” and “Security” in text are mapped to `department_id` 1, 5 and 8, which are Accounts, IT and Audit in the `departments` table. Any report built on the text column and any report built on the key will disagree.

CORRECTION — DEFECT 2

```
-- Detect the disagreement
SELECT
  e.employee_id,
  e.employee_name,
  e.department      AS department_text,
  d.department_name AS department_mapped
FROM employees e
JOIN departments d
  ON e.department_id = d.department_id
WHERE e.department <> d.department_name;

-- Resolution: drop the redundant text column and let the key be the single source of truth
ALTER TABLE employees
DROP COLUMN department;
```

Observation 3 — balances are static, not derived

SEVERITY: DESIGN STATUS: ROADMAP

`accounts.balance` is stored as a literal value and never adjusted by the rows in `transactions`, `fund_transfers` or `atm_transactions`. Account 1 shows ₹85,000 while its ledger records a ₹25,000 deposit and a ₹10,000 withdrawal. For a static reporting dataset this is fine, but a working system needs the two kept in step — either by recomputing the balance from the ledger, or by maintaining it inside a transaction with a trigger.

ROADMAP — DERIVING BALANCE FROM THE LEDGER

```
-- Recompute a balance from the ledger rather than trusting the stored column
SELECT
  a.account_id,
  a.account_number,
  a.balance AS stored_balance,
  a.balance
    + COALESCE(SUM(CASE WHEN t.transaction_type = 'Deposit' THEN t.amount END), 0)
    - COALESCE(SUM(CASE WHEN t.transaction_type = 'Withdrawal' THEN t.amount END), 0)
  AS ledger_balance
FROM accounts a
LEFT JOIN transactions t
  ON a.account_id = t.account_id
GROUP BY a.account_id, a.account_number, a.balance;
```

Observation 4 — a transfer is one row, not two ledger entries

SEVERITY: DESIGN STATUS: ROADMAP

`fund_transfers` records a movement as a single row with a sender and a receiver. Real double-entry accounting would post a debit against one account and a credit against the other, both inside one atomic transaction, so that the books always balance. The natural next iteration is a stored procedure that wraps both postings and the transfer record in a single `START TRANSACTION ... COMMIT` block, rolling back if either side fails.

ROADMAP — ATOMIC TRANSFER PROCEDURE

```
-- Sketch of the transactional version
DELIMITER $$

CREATE PROCEDURE TransferFunds(
  IN p_sender INT,
  IN p_receiver INT,
  IN p_amount DECIMAL(15,2),
  IN p_mode VARCHAR(30),
  IN p_ref VARCHAR(50)
)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  START TRANSACTION;

  UPDATE accounts SET balance = balance - p_amount WHERE account_id = p_sender;
  UPDATE accounts SET balance = balance + p_amount WHERE account_id = p_receiver;

  INSERT INTO fund_transfers
    (sender_account_id, receiver_account_id, amount, transfer_mode,
     transfer_status, reference_number)
  VALUES (p_sender, p_receiver, p_amount, p_mode, 'Completed', p_ref);

  COMMIT;
END $$

DELIMITER ;
```

Review Summary

Finding	Severity	Status	Action
Duplicate <code>fund_transfers</code> creation and load	Blocking	Fixed	Steps 21A and 21B removed
Department stored as both text and foreign key	Design	Documented	Drop the text column
Stored balance not derived from the ledger	Design	Roadmap	Trigger or derived view
Transfers not posted as double entries	Design	Roadmap	Transactional procedure

Results, Setup and Roadmap

What the project delivers, how to run it from a clean MySQL server, and where it goes next.

Key Findings from the Dataset

Question	Answer from the data
Largest deposit relationship	Manish Gupta — ₹432,000 across two accounts, including a ₹300,000 fixed deposit
Largest credit exposure	Ananya Shah — a ₹3.2 million home loan over 240 months
Strongest branch by deposits	Bandra Branch — ₹604,000 across five accounts
Loan book composition	₹27.3 million total; home loans are half of it at ₹13.7 million
Total repaid to date	₹358,500 across twenty loans, leaving ₹26.94 million outstanding
Busiest payment rail	UPI by count (6 transfers), RTGS by value (₹97,000)
Deposit vs withdrawal behaviour	Deposits average ₹48,733; withdrawals average ₹7,350
Payroll	₹1,142,000 per month across twenty employees in eight departments

Running the Project

The script assumes a clean MySQL 8.0 server and no existing `banking_system` schema. It must be executed in order, because every foreign key is enforced at creation time.

SETUP — COMMAND LINE

```
# 1. Start a MySQL client session
mysql -u root -p

# 2. Execute the script end to end
mysql -u root -p < banking_system.sql

# 3. Verify the build
mysql -u root -p -e "USE banking_system; SHOW TABLES;"
mysql -u root -p -e "USE banking_system; SHOW FULL TABLES WHERE Table_type = 'VIEW';"
mysql -u root -p -e "USE banking_system; SHOW PROCEDURE STATUS WHERE Db = 'banking_system';"

# 4. Try a procedure
mysql -u root -p -e "USE banking_system; CALL GetCustomerFinancialSummary(1);"
```

IF YOU ARE RE-RUNNING THE SCRIPT

Drop the schema first with `DROP DATABASE IF EXISTS banking_system;`. Re-running over an existing schema produces `ERROR 1050` on every `CREATE TABLE` and `ERROR 1062` on every insert with a unique key — the same class of failure documented in Section 09.

Suggested Repository Structure

REPOSITORY LAYOUT

```
banking-management-system/
├─ README.md           # overview, ER diagram, setup, sample output
├─ sql/
│   ├── 01_schema.sql   # Steps 1-15 : database, tables, ALTER
│   ├── 02_seed_master.sql # Steps 16-18 : branches, departments, customers
│   ├── 03_seed_operations.sql # Steps 19-28 : accounts through employees
│   ├── 04_views.sql     # Steps 29-38 : ten reporting views
│   ├── 05_analytics.sql  # Steps 39-51 : dashboard and ranking queries
│   └─ 06_procedures.sql  # Steps 50-63 : fourteen stored procedures
├─ docs/
│   ├── er-diagram.png
│   └─ project-documentation.pdf # this document
└─ screenshots/         # query output from MySQL Workbench
```

Splitting one long script into numbered files is what makes the project readable on GitHub. A reviewer can open `04_views.sql` and understand the reporting layer without scrolling past nine hundred lines of insert statements.

Complete Object Inventory

Type	Count	Objects
Database	1	banking_system
Tables	13	branches, customers, departments, accounts, loans, atms, employees, beneficiaries, transactions, fund_transfers, loan_payments, cards, atm_transactions
Views	10	customer_account_summary, loan_summary, transaction_summary, fund_transfer_summary, atm_transaction_summary, card_summary, branch_performance, employee_performance, loan_status_summary, transaction_status_summary
Procedures	14	GetCustomerLoans, GetCustomerLoanCount, GetCustomerLoanAmount, GetCustomerAccountCount, GetCustomerBalance, GetCustomerFinancialSummary, GetCustomerTransactionSummary, GetCustomerCardSummary, GetCustomerATMSummary, GetCustomerBeneficiarySummary, GetCustomerLoanStatusSummary, GetCustomerTransactionStatusSummary, GetCustomerFundTransferSummary, GetCustomerLoanPaymentSummary
Analytical queries	11	Top-5 reports, four category analyses, branch and department roll-ups, two <code>RANK()</code> league tables, one duplicate-detection check

Roadmap

Enhancement	Detail
Triggers	Adjust <code>accounts.balance</code> automatically on insert into <code>transactions</code> , removing the drift described in Section 09.
Transactional transfers	Wrap debit, credit and record in one <code>START TRANSACTION ... COMMIT</code> with a rollback handler.
EMI schedule	Generate a full amortisation schedule per loan from principal, rate and tenure rather than storing single ad-hoc payments.
Indexing	Add covering indexes on <code>transactions(account_id, transaction_date)</code> and <code>loan_payments(loan_id)</code> , then compare <code>EXPLAIN</code> output before and after.
Audit trail	An <code>audit_log</code> table capturing who changed what and when, populated by triggers.
Access control	Role-based MySQL users — teller, credit officer, auditor — each granted only the views their role needs rather than the base tables.
Visual layer	Point Power BI or Tableau at the reporting views to turn Section 07 into an interactive branch dashboard.

Skills Demonstrated

Taken together, this project covers the working vocabulary of relational database development: schema design and normalisation, referential integrity, constraint-driven data quality, inner and outer joins including self-referencing double joins, view-based abstraction, aggregate and window functions, procedural SQL with parameters and delimiters, and — in Section 09 — the habit of reviewing your own script critically and documenting what you would do differently.

END OF DOCUMENT

banking_system — 13 tables, 10 views, 14 stored procedures, 246 seeded rows. Every statement in the original script is reproduced in this document, together with the output it returns and the corrections applied.